

ĐẠI HỌC BÁCH KHOA HÀ NỘI
Trường Công nghệ Thông tin và Truyền thông

BÁO CÁO BÀI TẬP LỚN
Học phần: Nhập môn Trí tuệ nhân tạo

Mã học phần: IT3160 Mã lớp học: 167855

Đề tài:

Kỳ thủ Cờ Tướng nhân tạo

Giảng viên hướng dẫn:	TS. Ngô Văn Linh	
Nhóm sinh viên thực hiện:	Trần Tuấn Anh	202416124
	Lê Thành Trung	202400076
	Bùi Tiến Dũng	202416167

Hà Nội, Tháng 06 Năm 2026

Mục lục

1	Tổng quan đề tài	1
1.1	Đặt vấn đề	1
1.2	Phát biểu bài toán	1
1.2.1	Yêu cầu đầu vào và đầu ra	1
1.2.2	Yêu cầu hệ thống	2
1.3	Tổng quan về hệ thống Engine cờ tướng	2
1.4	Cách tiếp cận của nhóm chúng tôi	3
2	Phân tích hệ thống	4
2.1	Mô hình PEAS	4
2.2	Phân tích đặc tính môi trường	5
2.3	Kiến trúc hệ thống	5
2.3.1	Chức năng các tệp nguồn chính	6
3	Biểu diễn tri thức và Trạng thái	8
3.1	Định nghĩa không gian trạng thái	8
3.2	Kỹ thuật biểu diễn trạng thái trong Lingine	8
3.2.1	Biểu diễn bàn cờ bằng Bitboard 128-bit	10
3.2.2	Biểu diễn các thành phần lượt đi và lịch sử	13
3.3	Nén dữ liệu nước đi 16-bit	14
3.3.1	Ví dụ minh họa	15
3.4	Định danh trạng thái bằng Zobrist Hashing	15
3.4.1	Công thức băm	15
3.4.2	Khởi tạo bảng băm ngẫu nhiên tĩnh tại thời điểm biên dịch	16
3.4.3	Cập nhật vị phân trạng thái $O(1)$	16
3.5	Bộ phân xử luật cờ tướng và phát hiện lặp trạng thái	17
3.6	Thiết kế bảng chuyển vị Transposition Table	18
3.6.1	Cấu trúc dữ liệu tối ưu hóa bộ nhớ đệm	18
3.6.2	Chính sách thay thế của bảng chuyển vị	18
3.6.3	Chuyển đổi điểm số Mate phụ thuộc độ sâu (Mate Distance Pruning)	19
4	Mô hình tác tử và giải thuật	20
4.1	Mô hình chuyển đổi và Biểu diễn dữ liệu	20
4.1.1	Bitboards – Nền tảng tốc độ	20
4.1.2	Cơ chế sinh nước đi bằng Magic Bitboards	20
4.1.3	Sinh nước đi cho Xe và Pháo	22
4.2	Hàm đánh giá thế cờ	24
4.2.1	Công thức tổng quát	24

4.2.2	Nội suy pha ván cờ (Tapered Evaluation)	24
4.2.3	Tinh chỉnh tự động bằng Texel Tuning	25
4.2.4	Công thức chi tiết từng thành phần đánh giá	27
4.3	Chiến lược tìm kiếm	30
4.3.1	Thuật toán Alpha-Beta và Phân loại nút mạng	30
4.3.2	Quản lý thời gian	31
4.3.3	Sắp xếp nước đi	32
4.3.4	Tìm kiếm sâu dần (Iterative Deepening)	32
4.3.5	Cửa sổ kỳ vọng (Aspiration Windows & Fail-high Reductions)	33
4.3.6	Tìm kiếm biến thể chính (Principal Variation Search - PVS)	33
4.3.7	Các kỹ thuật Mở rộng không gian (Search Extensions)	34
4.3.8	Các kỹ thuật Cắt tỉa (Pruning) và Giảm độ sâu (Reductions)	35
4.3.9	Tìm kiếm tĩnh (Quiescence Search)	36
5	Trình diễn và Đánh giá Hệ thống	37
5.1	Kiểm thử bộ sinh nước đi bằng PERFT	37
5.2	Phương pháp đánh giá sức mạnh chơi cờ	38
5.2.1	Scripts và Môi trường Kiểm thử	38
5.2.2	Mô hình Ước lượng ELO bằng MLE	38
5.3	Lịch sử phát triển và Tiến trình tăng trưởng Elo	39
5.3.1	Bảng tổng hợp tiến trình 15 phiên bản	39
5.3.2	Phân tích các cột mốc lịch sử	39
5.3.3	Phân tích các Bước Nhảy ELO Lớn	41
5.4	Đánh giá chi tiết phiên bản mới nhất v1.9.0	42
5.4.1	Đánh giá Gauntlet (Phase 1)	42
5.4.2	Đối đầu trực tiếp phiên bản tiền nhiệm v1.8.0 (Phase 2)	42
5.4.3	Đối đầu trực tiếp phiên bản gốc v1.0.0 (Phase 3)	43
5.4.4	Đặc điểm Thống kê Đáng Chú Ý	43
6	Kết luận và Hướng phát triển	45
6.1	Các vấn đề thách thức và Giải pháp	45
6.2	Công nghệ và Tài nguyên sử dụng	45
6.3	Kết luận	46
6.4	Hướng phát triển tương lai	46
6.4.1	Tăng tính đa dạng trong lối chơi bằng Softmax Sampling	46
6.4.2	Tích hợp Mạng nơ-ron NNUE (Efficiently Updatable Neural Networks)	47
	Tài liệu tham khảo	48

Chương 1

Tổng quan đề tài

1.1 Đặt vấn đề

Từ thuở sơ khai của ngành Khoa học Máy tính, việc thiết kế các hệ thống trò chơi trí tuệ nhân tạo luôn là một trong những bài toán chuẩn mực nhằm đo lường khả năng tính toán và suy luận của máy móc. Trong khi Cờ vua được coi là “ruồi giấm” của AI phương Tây, thì Cờ tướng lại là trò chơi có chiều sâu văn hóa và lịch sử lâu đời, được hàng trăm triệu người yêu thích tại khu vực Đông Á, đặc biệt là Trung Quốc và Việt Nam.

Việc thiết kế và triển khai một engine cờ tướng đòi hỏi giải quyết các thách thức kỹ thuật lớn hơn rất nhiều so với cờ vua thông thường:

- **Kích thước bàn cờ lớn hơn:** Bàn cờ tướng có kích thước $10 \times 9 = 90$ ô (so với $8 \times 8 = 64$ ô của cờ vua).
- **Luật di chuyển đặc thù:** Sự hiện diện của Sông (hạn chế đường đi của Tượng và tăng cường sức mạnh của Tốt khi qua sông) và Cung (hạn chế phạm vi hoạt động của Sĩ và Tướng).
- **Cơ chế cản quân phức tạp:** Quân Mã bị cản chân mã, quân Tượng bị cản mắt tượng. Cơ chế này đòi hỏi thuật toán sinh nước đi phải xử lý các điều kiện biên động thay vì chỉ quét các tia tĩnh.
- **Luật lộ mặt Tướng:** Hai Tướng không được đối mặt trực tiếp trên cùng một cột dọc mà không có quân cờ nào che chắn ở giữa. Luật này tạo ra mối liên kết động rất mạnh giữa vị trí của hai vị Tướng trên toàn bộ cột dọc.

Chính vì những lý do này, việc tự tay xây dựng một engine cờ tướng hiệu năng cao từ con số 0 là cơ hội tuyệt vời để nghiên cứu sâu sắc các kỹ thuật lập trình hệ thống cấp thấp, tối ưu hóa thao tác bit trên CPU, cũng như triển khai các thuật toán tìm kiếm đối kháng hiện đại. Dự án của chúng tôi mang tên **Lingine** – một Engine Cờ Tướng hiệu năng cao được viết bằng ngôn ngữ lập trình hệ thống Rust hiện đại.

1.2 Phát biểu bài toán

1.2.1 Yêu cầu đầu vào và đầu ra

Lingine được thiết kế như một hệ thống xử lý độc lập hoạt động dưới dạng dòng lệnh, tuân thủ nghiêm ngặt giao thức truyền thông chuẩn UCI (Universal Chess Interface) đã được mở rộng dành riêng cho cờ tướng:

- **Đầu vào (Input):**

- *Trạng thái bàn cờ hiện tại:* Được biểu diễn dưới dạng chuỗi FEN (Forsyth-Edwards Notation) mô tả vị trí các quân cờ, lượt đi tiếp theo, số nước đi không ăn quân để tính luật 60 nước, số nước đi tổng cộng; hoặc chuỗi các nước đi lịch sử tính từ vị trí mặc định khởi đầu (`startpos moves ...`).
- *Các ràng buộc thời gian suy nghĩ:* Thời gian còn lại của hai bên tính bằng mili-giây, thời gian tích lũy cộng thêm sau mỗi nước đi, hoặc giới hạn độ sâu tìm kiếm cố định, v.v.
- *Các cấu hình tham số:* Kích thước bộ nhớ đệm bảng băm (Hash).

- **Đầu ra (Output):**

- *Nước đi tốt nhất tìm được:* Chuỗi ký tự biểu diễn nước đi tối ưu (ví dụ: `bestmove h2e2` thể hiện quân ở ô h2 di chuyển đến ô e2).
- *Thông tin phân tích thời gian thực:* Được in ra luồng Standard Output định kỳ gồm độ sâu tìm kiếm (`depth`), điểm số đánh giá (`score cp` hoặc `score mate`), số nút đã duyệt (`nodes`), tốc độ duyệt (`nps`), thời gian đã dùng (`time`), và biên thể chính – dãy nước đi tốt nhất dự tính (`pv`).

1.2.2 Yêu cầu hệ thống

Để đảm bảo engine có khả năng thi đấu thực tế và mang lại trải nghiệm phân tích tốt nhất, hệ thống cần đáp ứng ba tiêu chuẩn vận hành:

1. **Tính chính xác tuyệt đối:** Mọi nước đi sinh ra phải tuyệt đối hợp lệ theo luật cờ tướng quốc tế và luật cờ tướng Việt Nam. Engine phải phát hiện chính xác các tình huống chiếu tướng, cản quân, chiếu bí, và các lỗi lặp nước đi hoặc chiếu dai.
2. **Độ trễ thấp và phản hồi thời gian thực:** Hệ thống phải có cơ chế quản lý thời gian thông minh để phân bổ thời gian hợp lý, đưa ra nước đi tốt nhất trước khi hết giờ và phản ứng tức thì khi nhận được lệnh dừng (`stop`) từ người dùng.
3. **Hiệu suất tìm kiếm vượt trội:** Tận dụng tối đa sức mạnh phần cứng thông qua việc tối ưu mã nguồn, tìm kiếm được nhiều vị trí tốt nhất có thể trong thời gian giới hạn.

1.3 Tổng quan về hệ thống Engine cờ tướng

Một engine cờ tướng hiện đại luôn được tổ chức theo kiến trúc ba thành phần cốt lõi tương tác chặt chẽ:

1. **Biểu diễn trạng thái (Board Representation):** Chịu trách nhiệm lưu trữ cấu trúc bàn cờ, vị trí quân, lịch sử ván đấu để hỗ trợ thực hiện nước đi (`do_move`) và hoàn tác nước đi (`undo_move`).
2. **Bộ sinh nước đi (Move Generator):** Chịu trách nhiệm sinh ra toàn bộ các nước đi giả hợp lệ (pseudo-legal) và lọc ra các nước đi hợp lệ (legal) tại vị trí hiện tại. Đây là phần lõi tính toán được gọi hàng triệu lần mỗi giây, yêu cầu tối ưu hóa tốt nhất có thể.

3. **Bộ tìm kiếm và đánh giá (Search & Evaluation):** Áp dụng giải thuật duyệt cây đối kháng Alpha-Beta với biến thể Negamax để tìm nước đi tối ưu. Tại các nút lá của cây tìm kiếm, hàm đánh giá ước lượng điểm của thế cờ của hai bên.

1.4 Cách tiếp cận của nhóm chúng tôi

Trong dự án này, nhóm phát triển lựa chọn hướng tiếp cận kết hợp giữa các cấu trúc dữ liệu tiên tiến và các thuật toán tìm kiếm tối ưu hóa hiệu năng cao:

- **Ngôn ngữ lập trình Rust:** Rust mang lại hiệu năng tối đa của ngôn ngữ cấp thấp (như C/C++) mà không gặp các lỗi bảo mật bộ nhớ nguy hiểm. Trình biên dịch Rust áp dụng các tối ưu hóa mạnh mẽ tại thời điểm biên dịch, giúp nâng cao đáng kể tốc độ thực thi của engine.
- **Cấu trúc dữ liệu Bitboard 128-bit:** Khác với các engine truyền thống sử dụng mảng 2 chiều 90 ô gây chậm trễ khi truy xuất, chúng tôi sử dụng cấu trúc Bitboard lưu trữ trong biến nguyên 128-bit để đại diện cho bàn cờ tướng 10×9 . Các phép toán logic bit giúp tính toán các luồng tấn công và nước đi của toàn bộ bàn cờ cùng lúc chỉ với vài chu kỳ CPU.
- **Magic Bitboards cho các quân cản như Tượng, Mã:** Áp dụng kỹ thuật Magic Multiplication 128-bit để tra bảng nhanh các nước đi bị chặn của Tượng và Mã, thay vì phải chạy vòng lặp kiểm tra từng ô cản.
- **Hàm đánh giá Tapered Evaluation tinh chỉnh bằng Texel Tuning:** Chúng tôi phân tách điểm số đánh giá thành hai pha: khai/trung cuộc và tàn cuộc, sau đó nội suy tuyến tính theo pha ván cờ. Toàn bộ trọng số quân cờ và bảng vị trí từng quân cờ được huấn luyện tự động bằng phương pháp Texel Tuning trên tập dữ liệu hàng triệu thế cờ.
- **Chiến lược tìm kiếm Negamax tối ưu nâng cao:** Kết hợp tìm kiếm biến thể chính, cửa sổ kỳ vọng, bảng chuyển vị, và hàng loạt kỹ thuật tỉa nhánh hiện đại để giảm thiểu tối đa số lượng nút cần duyệt mà vẫn giữ nguyên độ sâu chiến thuật.

Chương 2

Phân tích hệ thống

2.1 Mô hình PEAS

Để đặc tả bài toán cờ tướng dưới góc nhìn của một tác tử thông minh, chúng ta áp dụng mô hình PEAS bao gồm bốn thành phần: Performance (Độ đo hiệu suất), Environment (Môi trường), Actuators (Bộ phận chấp hành), và Sensors (Bộ phận cảm biến).

- **Performance (Độ đo hiệu suất):**

- Mục tiêu tối cao là chiến thắng ván đấu thông qua việc chiếu hết hoặc dồn đối thủ vào thế hết nước đi – khác với cờ vua, trong cờ tướng hết nước đi được tính là thua.
- Tối ưu hóa quỹ thời gian suy nghĩ, không để xảy ra trường hợp bị xử thua do hết giờ. **Nodes Per Second (NPS):** Tốc độ duyệt số nút trên cây trò chơi trên giây phản ánh trực tiếp hiệu năng phần mềm và khả năng khai thác luồng tính toán của CPU.

- **Environment (Môi trường):**

- Bàn cờ tướng kích thước $10 \times 9 = 90$ ô vuông với 32 quân cờ chia đều cho hai bên Đỏ và Đen.
- Đồng hồ thi đấu quy định thời gian suy nghĩ cố định và thời gian tích lũy.
- Đối thủ (có thể là con người sử dụng giao diện đồ họa hoặc một engine AI khác thông qua giao thức UCI).

- **Actuators (Bộ phận chấp hành):**

- Tác động lên môi trường bằng cách gửi các nước đi dưới dạng chuỗi văn bản (ví dụ: `bestmove h2e2`) thông qua luồng đầu ra tiêu chuẩn.
- Truyền tải thông tin phân tích thế cờ (`info`) để hiển thị trên giao diện người dùng.

- **Sensors (Bộ phận cảm biến):**

- Nhận biết trạng thái môi trường thông qua luồng đầu vào tiêu chuẩn (`Stdin`).
- Tiếp nhận chuỗi FEN hoặc chuỗi nước đi lịch sử mô tả trạng thái hiện tại của bàn cờ và các tham số thời gian từ lệnh `go`.

2.2 Phân tích đặc tính môi trường

Bài toán cờ tướng sở hữu các đặc tính môi trường chuẩn mực của một trò chơi đối kháng cổ điển:

- **Quan sát hoàn toàn (Fully Observable):** Trạng thái bàn cờ hoàn toàn công khai tại mọi thời điểm. Vị trí của tất cả các quân cờ trên bàn cờ đều hiển thị rõ ràng, không có yếu tố thông tin ẩn hay ngẫu nhiên.
- **Xác định và mang tính chiến lược (Deterministic – Strategic):** Trạng thái tiếp theo của bàn cờ được quyết định tuyệt đối bởi nước đi hiện tại, không có yếu tố may rủi. Tuy nhiên, hành vi của đối thủ là không thể kiểm soát trước, đòi hỏi tác tử phải tính toán chiến lược đối phó tối ưu trước mọi nước đi có thể xảy ra của đối phương.
- **Liên tiếp (Sequential):** Mỗi quyết định đi quân ở hiện tại đều ảnh hưởng sâu sắc đến thế trận trong tương lai. Lịch sử di chuyển cũng quyết định các quy tắc hòa cờ, lặp nước đi hoặc chiếu dai.
- **Bán động (Semi-dynamic):** Môi trường bàn cờ vật lý không thay đổi trong khi tác tử suy nghĩ, nhưng thời gian thi đấu vẫn trôi qua liên tục trên đồng hồ. Nếu tác tử suy nghĩ quá lâu, nó sẽ mất lợi thế về thời gian hoặc bị xử thua trực tiếp.
- **Rời rạc (Discrete):** Không gian trạng thái là rời rạc. Chỉ có tối đa 90 ô cờ, số lượng quân cờ hữu hạn (32), và số lượng nước đi hợp lệ tại mỗi thế cờ là một tập hợp hữu hạn đếm được (thường không quá 120 nước đi).
- **Đa tác tử đối kháng (Multi-agent):** Hai tác tử tham gia trực tiếp đối đầu (Đỏ và Đen). Trò chơi mang tính chất tổng bằng không, lợi thế của bên này đồng nghĩa với sự suy giảm cơ hội của bên kia.

2.3 Kiến trúc hệ thống

Lingine áp dụng kiến trúc 2 luồng song song tối giản nhưng cực kỳ hiệu quả nhờ tận dụng cơ chế quản lý luồng hiện đại của ngôn ngữ lập trình Rust.

Sơ đồ hoạt động của hệ thống được mô tả như sau:

1. Luồng A – Luồng lắng nghe (Stdin Listener Thread):

- Hoạt động độc lập, thực hiện nhiệm vụ duy nhất là đọc liên tục các dòng lệnh văn bản từ `stdin`.
- Phân tích nhanh các lệnh điều khiển. Nếu nhận được lệnh dừng khẩn cấp (`stop`) hoặc thoát chương trình (`quit`), Luồng A ngay lập tức đặt cờ hiệu nguyên tử toàn cục `keep_running` thành `false`.
- Các lệnh khác (như thiết lập thế cờ, cấu hình bảng băm) được đẩy vào một kênh truyền thông điệp bất đồng bộ để chuyển giao cho Luồng B.

2. Luồng B – Luồng xử lý chính và Tìm kiếm (Main Search Thread):

- Đóng vai trò là trung tâm điều phối của Engine. Nó nhận thông điệp từ kênh truyền để cập nhật vị trí bàn cờ hiện tại (**Position**) hoặc cấu hình các thông số.
- Khi nhận lệnh tìm kiếm (**go**), Luồng B sẽ thiết lập cờ **keep_running** thành **true** và khởi chạy thuật toán tìm kiếm Negamax sâu dần.
- Trong vòng lặp Negamax tại mỗi nút trên cây trò chơi, Luồng B định kỳ kiểm tra giá trị của **keep_running**. Nếu cờ chuyển sang **false**, quá trình tìm kiếm lập tức bị hủy bỏ và thoát ra ngoài, trả về nước đi tốt nhất hiện tại.

Kiến trúc 2 luồng này giúp giảm thiểu đáng kể overhead đồng bộ dữ liệu, tránh tình trạng giằng co khi khóa mutex, đồng thời đảm bảo thời gian phản hồi lệnh ngắt **stop** gần như tức thì.

2.3.1 Chức năng các tệp nguồn chính

Mã nguồn của Lingine được cấu trúc mạch lạc thành các mô-đun chức năng độc lập trong thư mục **src/**:

- **Thư mục gốc **src/**:**
 - **main.rs**: Điểm khởi chạy chương trình, khởi tạo kiến trúc 2 luồng và khởi động vòng lặp lệnh UCI chính.
 - **lib.rs**: Khai báo các mô-đun công khai cho toàn bộ thư viện.
- **Mô-đun Core (**src/core/**):** Định nghĩa các cấu trúc dữ liệu nền tảng.
 - **bitboard.rs**: Triển khai cấu trúc Bitboard dựa trên biến nguyên 128-bit để đại diện cho bàn cờ 10×9 , kèm theo các phép toán thao tác bit tối ưu.
 - **types/**: Định nghĩa các kiểu thực thể cơ bản như ô cờ (**Square**), quân cờ (**Piece**), loại quân (**PieceType**), bên đi (**Side**), nước đi (**Move**), và điểm số (**Score**).
 - **mod.rs** (trong **position/**): Cấu trúc dữ liệu trung tâm **Position** quản lý toàn bộ trạng thái bàn cờ, cập nhật thế cờ và hoàn tác nước đi (**do_move/undo_move**).
 - **attacks.rs** (trong **position/**): Xác định xem một ô cờ có đang bị tấn công bởi đối phương hay không (phục vụ kiểm tra tính hợp lệ của nước đi).
 - **rule_judge.rs** (trong **position/**): Trọng tài luật của ván cờ, xử lý luật 60 nước đi không ăn quân, luật lặp lại thế trận và chiếu dai.
 - **movegen/mod.rs**: Triển khai bộ sinh nước đi giả hợp lệ (pseudo-legal) và nước đi hợp lệ (legal) cho tất cả các loại quân.
 - **movegen/queries.rs**: Các truy vấn tấn công nhanh của quân cờ tận dụng các bảng tra cứu tĩnh và ma thuật.
- **Mô-đun Đánh giá (**src/eval/**):** Hàm lượng hóa ưu thế thế cờ.
 - **mod.rs**: Tính toán tổng hợp điểm số thế cờ kết hợp các thành phần.
 - **piece_material_value.rs**: Định nghĩa giá trị vật chất của các quân cờ trong hai pha Middlegame và Endgame.

- `piece_square_tables.rs`: Bảng vị trí quân cờ quy định mức độ ưu tiên đứng của từng loại quân trên bàn cờ.
- `defender_bonus.rs`: Điểm thưởng khi tồn tại quân cờ hỗ trợ bảo vệ lẫn nhau (đặc biệt là Sĩ, Tượng bảo vệ Tướng).
- `mobility_tables.rs`: Bảng điểm thưởng theo mức độ cơ động (số nước đi khả dĩ) của Xe, Pháo, Mã.
- **Mô-đun Tìm kiếm (`src/search/`):** Triển khai trí tuệ nhân tạo duyệt cây đối kháng.
 - `mod.rs`: Quản lý ngữ cảnh tìm kiếm và điều phối luồng tìm kiếm chính.
 - `negamax.rs`: Giải thuật Negamax Alpha-Beta làm xương sống cho quá trình duyệt cây.
 - `pv_search.rs`: Kỹ thuật tìm kiếm biến thể chính.
 - `aspiration_search.rs`: Kỹ thuật tìm kiếm với cửa sổ kỳ vọng hẹp để tối ưu hóa phạm vi Alpha-Beta.
 - `selectivity.rs`: Tập hợp các kỹ thuật cắt tỉa nhánh và gia tăng độ sâu.
 - `transposition_table.rs`: Triển khai bảng chuyển vị (bảng băm ghi nhớ các trạng thái đã duyệt).
 - `move_ordering.rs`: Bộ sắp xếp nước đi dựa để tối ưu hóa tốc độ cắt tỉa Alpha-Beta.
- **Mô-đun UCI (`src/uci/`):**
 - `command.rs`: Phân tích cú pháp chuỗi lệnh nhận từ stdin.
 - `engine.rs`: Quản lý trạng thái kết nối và phản hồi UCI.
 - `output.rs`: Định dạng và xuất dữ liệu phân tích ra stdout theo đúng chuẩn giao thức.

Chương 3

Biểu diễn tri thức và Trạng thái

3.1 Định nghĩa không gian trạng thái

Trạng thái toàn cục S của một ván cờ tướng tại một thời điểm được định nghĩa chính thức bởi bộ bốn thành phần:

$$S = \langle B, p, c_h, h \rangle$$

Trong đó:

- B (Board) đại diện cho phân bố các quân cờ trên bàn lưới kích thước $10 \times 9 = 90$ ô. Mỗi ô cờ tại vị trí $sq \in [0, 89]$ có thể trống hoặc chứa một trong 14 loại quân cờ thực tế (Tướng, Sĩ, Tượng, Xe, Pháo, Mã, Tốt cho mỗi bên Đỏ và Đen).
- p (Player) đại diện cho bên đi tiếp theo, $p \in \{\text{Đỏ}, \text{Đen}\}$.
- c_h (Count Half-move) là số nửa nước đi trôi qua kể từ nước đi không có bắt quân cuối cùng. Đây là bộ đếm phục vụ cho việc thực thi luật hòa cờ 60 nước.
- h (History) là danh sách lịch sử lưu trữ toàn bộ trạng thái di chuyển trước đó cùng các thông tin bắt quân, hỗ trợ hoàn tác nước đi và phát hiện các trường hợp lặp lại trạng thái.

3.2 Kỹ thuật biểu diễn trạng thái trong Lingine

Để hiện thực hóa mô hình toán học trên với hiệu năng cao nhất, Lingine định nghĩa cấu trúc dữ liệu trung tâm `Position` và cấu trúc thông tin phụ trợ `StateInfo` trong tệp `src/core/position/mod.rs`:

```
1 /// Encapsulates the complete game board representation, bitboards, turn
2 /// tracking, ply count, and incremental state histories for the engine.
3 #[derive(Clone)]
4 pub struct Position {
5     /// Flat 90-square array mapping square index (0 to 89) to the Piece
6     /// occupying it.
7     board: [Option<Piece>; Square::COUNT], // 1 * 90 = 90 bytes
8     /// Precomputed bitboards showing piece placements grouped by `
9     PieceType`.
10    bitboard_by_type: [Bitboard; PieceType::COUNT], // 7 * 16 = 112 bytes
11    /// Precomputed bitboards showing piece placements grouped by `Color`.
12    bitboard_by_color: [Bitboard; Side::COUNT], // 2 * 16 = 32 bytes
13    /// Active count of each piece category on the board.
```

```

13 piece_count: [u8; Piece::COUNT], // 15 bytes
14 /// Current state of position
15 state: StateInfo, // 224 bytes
16 /// Stack tracking previous move parameter histories for undoing moves.
17 history: Vec<StateInfo>, // ptr (8) + size (8) + cap(8) = 24 bytes
18 /// Total moves played in the game so far (Red = 0, Black = 1, Red's
19 /// next = 2, etc.).
20 game_ply: u16, // 2 bytes
21 /// Palace coordinates of both players' Generals (Kings) for faster
check
22 /// detection.
23 king_squares: [Square; Side::COUNT], // 2 bytes
24 }

```

Listing 3.1: Cấu trúc Position trong Lngine

Để tiết kiệm thời gian tính toán lại thế cờ trong quá trình tìm kiếm sâu, các dữ liệu động phát sinh sau mỗi nước đi được đóng gói riêng vào cấu trúc `StateInfo`:

```

1 /// Represents search state parameters that must be saved on a stack,
2 /// allowing incremental undo_move restorations.
3 #[derive(Clone, Copy, Debug)]
4 struct StateInfo {
5     /// The last move played to reach this state.
6     pub last_move: Option<Move>,
7     /// The piece captured during this move, or `None` if it was quiet.
8     pub captured_piece: Option<Piece>,
9     /// The prior Zobrist position hash value before the move occurred.
10    pub zobrist: u64,
11    /// Halfmove clock / 60-rule counter (increments on quiet moves, resets
to 0
12    /// on captures/pawn moves).
13    pub sixtymove_clock: u16,
14    /// Whether the side to move was in check in this position state.
15    pub in_check: bool,
16    /// Precalculated incremental mid- and end-game score (from Red's
17    /// perspective)
18    pub score: PackedScore,
19    /// Precalculated incremental game phase
20    pub phase: u8,
21    /// Checker pieces checking the King of the side to move.
22    pub checkers: Bitboard,
23    /// Pinned pieces (blockers) for both sides' kings.
24    pub blockers_for_king: [Bitboard; Side::COUNT],
25    /// The slider pieces pinning other pieces for both sides' kings.
26    pub pinners: [Bitboard; Side::COUNT],
27    /// Check squares for each piece type of the side to move (against the
28    /// opponent king).
29    pub check_squares: [Bitboard; PieceType::COUNT],
30    /// Whether we need a full check validation.
31    pub need_full_check: bool,
32    /// Number of plies played since the last null move.
33    pub plies_since_null: u16,
34 }

```

Listing 3.2: Cấu trúc StateInfo trong Lngine

3.2.1 Biểu diễn bàn cờ bằng Bitboard 128-bit

Do bàn cờ tướng có kích thước $10 \times 9 = 90$ ô, cấu trúc Bitboard chuẩn 64-bit của cờ vua ($8 \times 8 = 64$ ô) là hoàn toàn không đủ. Lengine đã phát triển một giải pháp Bitboard dựa trên kiểu số nguyên không dấu 128-bit (u128), trong đó 90 bit đầu tiên được gán tương ứng cho 90 ô cờ từ vị trí 0 (a1) tới 89 (i10):

$$\text{Bitboard} = \sum_{sq=0}^{89} b_{sq} \cdot 2^{sq} \quad (\text{với } b_{sq} \in \{0, 1\})$$

Cấu trúc Bitboard được cài đặt trong tệp [src/core/bitboard.rs](#) như sau:

```

1 #[derive(
2     BitAnd,
3     BitAndAssign,
4     BitOr,
5     BitOrAssign,
6     BitXor,
7     BitXorAssign,
8     Not,
9     Clone,
10    Copy,
11    PartialEq,
12    Eq,
13    Debug,
14    Default,
15 )]
16 pub struct Bitboard(u128);
17
18 impl Bitboard {
19     /// Creates a new, completely empty bitboard (all bits set to 0).
20     #[inline]
21     pub const fn new() -> Self {
22         Self(0)
23     }
24
25     /// Make a bitboard directly from a raw `u128` value.
26     ///
27     /// # Safety
28     ///
29     /// The caller must ensure that the provided `u128` value only has
30     /// bits set in the range 0..89, as bits 90 to 127 are unused and
31     /// may lead to undefined behavior
32     #[inline]
33     pub const unsafe fn from_raw(bitboard: u128) -> Self {
34         Self(bitboard)
35     }
36
37     /// Retrieves the raw `u128` value of the bitboard, useful for
38     /// low-level operations or debugging.
39     #[inline]
40     pub const fn raw(&self) -> u128 {
41         self.0
42     }
43
44     /// Checks if the bitboard is completely empty (no active squares).
45     #[inline]

```

```

46 pub const fn is_empty(&self) -> bool {
47     self.0 == 0
48 }
49
50 /// Verifies if a given square bit is active (set to 1) in the bitboard
51 .
52 #[inline]
53 pub const fn is_occupied(&self, square: Square) -> bool {
54     (self.0 & (1u128 << (square as u8))) != 0
55 }
56
57 /// Checks if this board (set of squares) contains the square
58 #[inline]
59 pub const fn contains(&self, square: Square) -> bool {
60     self.is_occupied(square)
61 }
62
63 /// Activates (sets to 1) the bit corresponding to the given square.
64 #[inline]
65 pub const fn set_bit(&mut self, square: Square) {
66     self.0 |= 1u128 << (square as u8);
67 }
68
69 /// Deactivates (clears to 0) the bit corresponding to the given square
70 .
71 #[inline]
72 pub const fn clear_bit(&mut self, square: Square) {
73     self.0 &= !(1u128 << (square as u8));
74 }
75
76 /// Returns the number of active squares (bits set to 1) in the
77 bitboard.
78 #[inline]
79 pub const fn count_ones(&self) -> u32 {
80     self.0.count_ones()
81 }
82
83 /// Pops (extracts and clears) the Least Significant Bit (LSB) from
84 the bitboard, returning the corresponding `Square`. Returns
85 `None` if the bitboard is empty. Used for rapid, zero-overhead
86 sequence generation and iterator-like popping of pieces.
87 #[inline]
88 pub const fn pop_lsb(&mut self) -> Option<Square> {
89     if self.0 == 0 {
90         None
91     } else {
92         let lsb = self.0.trailing_zeros() as u8;
93         self.0 &= self.0 - 1; // Clears the lowest set bit
94         Some(unsafe { Square::from_repr_unchecked(lsb) })
95     }
96 }
97
98 /// A const-compatible bitwise OR operator for building compile-time
99 constants lookup tables.
100 #[inline]
101 pub const fn const_or(&self, b: Self) -> Self {
102     Self(self.0 | b.0)
103 }

```

```

101
102     /// A const-compatible bitwise AND operator for building compile-time
103     /// constants
104     #[inline]
105     pub const fn const_and(&self, b: Self) -> Self {
106         Self(self.0 & b.0)
107     }
108
109     #[inline]
110     pub const fn from_square(s: Square) -> Self {
111         Self(1u128 << (s as u8))
112     }
113
114     /// Constructs a bitboard with all bits active along the specified
115     /// vertical column (`File`).
116     #[inline]
117     pub const fn from_file(f: File) -> Self {
118         let mut bits = 0u128;
119         let mut r = 0u8;
120         let f = f as u8;
121
122         while r < Rank::COUNT as u8 {
123             let square_index = (r * File::COUNT as u8) + f;
124             bits |= 1u128 << square_index;
125             r += 1;
126         }
127
128         Self(bits)
129     }
130
131     /// Constructs a bitboard with all bits active along the specified
132     /// horizontal row (`Rank`).
133     #[inline]
134     pub const fn from_rank(r: Rank) -> Self {
135         Self(0x1FFu128 << (r as u8 * 9))
136     }
137
138     /// Returns a precomputed bitboard covering one half of the board
139     /// (5 ranks) for a given player:
140     #[inline]
141     pub const fn side(side: Side) -> Self {
142         match side {
143             Side::Red => Self::from_rank(Rank::R0)
144                 .const_or(Self::from_rank(Rank::R1))
145                 .const_or(Self::from_rank(Rank::R2))
146                 .const_or(Self::from_rank(Rank::R3))
147                 .const_or(Self::from_rank(Rank::R4)),
148             Side::Black => Self::from_rank(Rank::R5)
149                 .const_or(Self::from_rank(Rank::R6))
150                 .const_or(Self::from_rank(Rank::R7))
151                 .const_or(Self::from_rank(Rank::R8))
152                 .const_or(Self::from_rank(Rank::R9)),
153         }
154     }
155
156     /// Returns a precomputed bitboard covering the palace of the given
157     /// player
158     #[inline]

```

```

158 pub const fn palace(side: Side) -> Self {
159     let ranks = match side {
160         Side::Red => Self::from_rank(Rank::R0)
161             .const_or(Self::from_rank(Rank::R1))
162             .const_or(Self::from_rank(Rank::R2)),
163         Side::Black => Self::from_rank(Rank::R7)
164             .const_or(Self::from_rank(Rank::R8))
165             .const_or(Self::from_rank(Rank::R9)),
166     };
167
168     ranks.const_and(
169         Self::from_file(File::FD)
170             .const_or(Self::from_file(File::FE))
171             .const_or(Self::from_file(File::FF)),
172     )
173 }
174
175 pub const PALACE: Self = Self::palace(Side::Red).const_or(Self::palace(
176     Side::Black));

```

Listing 3.3: Cấu trúc Bitboard trong Lngine

Việc sử dụng u128 cho phép CPU thực hiện các phép tính logic bit song song trên toàn bộ 90 ô cờ chỉ bằng một chỉ thị máy duy nhất. Nhờ cơ chế `pop_lsb` sử dụng lệnh assembly `TZCNT` của vi xử lý x86-64 thông qua hàm `trailing_zeros()`, việc duyệt qua tất cả các quân cờ đang hoạt động đạt tốc độ tối đa.

Trong `Position`, ta lưu thành phần B bằng các biến:

- **board**: Mảng 90 phần tử chứa quân cờ hiện tại ở mỗi vị trí trên bàn cờ.
- **bitboard_by_type**: Mảng có 7 phần tử tương ứng với 7 quân cờ, mỗi phần tử là một Bitboard thể hiện vị trí của quân cờ đó ở thế cờ hiện tại.
- **bitboard_by_color**: Mảng có 2 phần tử tương ứng với 2 người chơi, mỗi phần tử là một Bitboard thể hiện vị trí của quân cờ của người chơi đó ở thế cờ hiện tại.

3.2.2 Biểu diễn các thành phần lượt đi và lịch sử

- **Lượt đi (p)**: Được biểu diễn bằng kiểu `Side` gồm hai giá trị `Side::Red` và `Side::Black`. Các phương thức đối lập `Side::opposite()` giúp đổi lượt nhanh bằng phép toán số học đơn giản.
- **Bộ đếm lượt hòa cờ (c_h)**: Được lưu trong trường `sixtymove_clock`: u16. Mỗi nước đi quiet (không ăn quân) sẽ tăng bộ đếm lên 1. Nếu có quân bị bắt, bộ đếm lập tức reset về 0. Khi giá trị đạt 120 (tương đương 60 nước đi đầy đủ của cả hai bên), engine sẽ nhận diện thế hòa.
- **Lịch sử (h)**: Lưu trữ trong `stack history`: `Vec<StateInfo>`. Mỗi khi thực hiện nước đi thông qua `do_move()`, trạng thái `StateInfo` cũ được đẩy vào `history`. Khi hoàn tác qua `undo_move()`, trạng thái chỉ cần pop từ stack ra để khôi phục nhanh chóng trong thời gian $O(1)$ mà không cần tính toán lại từ đầu.

3.3 Nén dữ liệu nước đi 16-bit

Trong quá trình duyệt cây tìm kiếm, hàng triệu nước đi được tạo ra và truyền qua lại giữa các hàm. Nếu sử dụng các cấu trúc dữ liệu công kênh, chi phí cấp phát bộ nhớ và truyền tham số sẽ làm chậm đáng kể tốc độ của engine. Lengine giải quyết triệt để vấn đề này bằng cách nén nước đi thành một số nguyên không dấu 16-bit (u16) duy nhất:

Các bit	Khoảng giá trị	Ý nghĩa
0 – 6	0 .. 89	Chỉ số ô đích (To Square) – Cần 7 bit ($2^7 = 128$)
7 – 13	0 .. 89	Chỉ số ô xuất phát (From Square) – Cần 7 bit
14 – 15	0 .. 3	Không được sử dụng

Bảng 3.1: Cấu trúc mã hóa nước đi 16-bit trong Lengine

Cấu trúc Move được cài đặt trong tệp `src/core/types/move.rs` như sau:

```

1 /// A compact 16-bit move representation designed for performance:
2 ///
3 /// * **Bits 0 - 6**:: Destination square (0 to 89, fits in 7 bits since
4 ///   $2^7=128$).
5 /// * **Bits 7 - 13**:: Origin square (0 to 89, fits in 7 bits).
6 /// * **Bits 14 - 15**:: Reserved.
7 ///
8 /// The move is guaranteed to be non-zero, so we can use NonZeroU16 for
9 /// better optimization when using Option<Move>.
10 #[derive(Debug, Clone, Copy, PartialEq, Eq)]
11 pub struct Move(NonZeroU16);
12
13 impl Move {
14     /// Constructs a basic quiet or capture move from an origin and
15     /// destination square.
16     #[inline]
17     pub const fn new(from: Square, to: Square) -> Self {
18         assert!(
19             from as u16 != to as u16,
20             "Move cannot have the same origin and destination square"
21         );
22         Self(unsafe { NonZeroU16::new_unchecked(((to as u16) | ((from as u16
23 ) << 7)) ) })
24     }
25
26     /// Extracts the starting square index by shifting past the
27     /// destination bits.
28     #[inline]
29     pub const fn from(&self) -> Square {
30         unsafe { std::mem::transmute(((self.0.get() >> 7) & 0x7F) as u8) }
31     }
32
33     /// Extracts the target square index by masking the lower 7 bits.
34     #[inline]
35     pub const fn to(&self) -> Square {
36         unsafe { std::mem::transmute((self.0.get() & 0x7F) as u8) }
37     }
38
39     /// Converts the move into its UCI string format

```

```

39 pub fn to_uci_string(&self) -> String {
40     let from = self.from();
41     let to = self.to();
42     let from_file = (b'a' + from.file() as u8) as char;
43     let from_rank = (b'0' + from.rank() as u8) as char;
44     let to_file = (b'a' + to.file() as u8) as char;
45     let to_rank = (b'0' + to.rank() as u8) as char;
46     format!("{}", from_file, from_rank, to_file, to_rank)
47 }
48 }

```

Listing 3.4: Cấu trúc Move trong Lengine

3.3.1 Ví dụ minh họa

Giả sử ta cần mã hóa nước đi di chuyển quân Xe từ ô xuất phát *h2* (chỉ số 16, nhị phân 0010000) đến ô đích *e2* (chỉ số 13, nhị phân 0001101):

- Chỉ số ô đi (From): $16 \ll 7 = 2048$ (nhị phân 00001000 00000000)
- Chỉ số ô đến (To): 13 (nhị phân 00000000 00001101)
- Giá trị nước đi mã hóa: $\text{Move} = 2048 \vee 13 = 2061$ (thập lục phân 0x080D)

Nhờ kích thước cực nhỏ (2 bytes), dữ liệu nước đi có thể nằm gọn trong các thanh ghi CPU và truyền trực tiếp bằng trị số, tránh tối đa việc truy xuất bộ nhớ RAM và tận dụng tối ưu bộ nhớ đệm L1 Cache.

3.4 Định danh trạng thái bằng Zobrist Hashing

Mã băm Zobrist (Zobrist Hashing) là một kỹ thuật giúp chuyển đổi một thế cờ có kích thước bất kỳ thành một dãy số có độ dài cố định, với tỷ lệ phân bố đồng đều trên tất cả các giá trị có thể xảy ra, được phát minh bởi Albert Zobrist.

Mục đích chính của Zobrist Hashing trong Lengine là tạo ra một chỉ số gần như độc nhất cho bất kỳ thế cờ nào, đi kèm một yêu cầu rất quan trọng: hai thế cờ tương tự nhau phải tạo ra hai chỉ số hoàn toàn khác biệt. Các chỉ số này được sử dụng để giúp các bảng băm hoạt động nhanh hơn và tiết kiệm không gian lưu trữ hơn.

Trong Lengine, mã băm Zobrist Hashing giúp kiểm tra lặp thế cờ cực nhanh và làm khóa tra bảng chuyển vị.

3.4.1 Công thức băm

Mã băm của trạng thái thế cờ S_0 được tính bằng:

$$\text{Hash}(S_0) = Z_{\text{side}} \oplus \left(\bigoplus_{sq=0}^{89} Z_{\text{piece}(sq),sq} \right)$$

Trong đó:

- Z_{side} là 0 nếu bên đi là Đỏ, hoặc một giá trị ngẫu nhiên 64-bit nếu bên đi là Đen.
- $Z_{\text{piece}(sq),sq}$ là giá trị ngẫu nhiên 64-bit tương ứng với quân cờ đang ngự trị tại ô sq . Nếu ô sq trống thì giá trị này bằng 0.

3.4.2 Khởi tạo bảng băm ngẫu nhiên tĩnh tại thời điểm biên dịch

Để đảm bảo tính nhất quán và tốc độ tối đa, toàn bộ bảng mã ngẫu nhiên Zobrist được tính toán trực tiếp tại thời điểm biên dịch (Compile-time) thông qua cơ chế `const fn` của Rust. Nhóm phát triển đã chọn Seed cho bộ sinh số ngẫu nhiên dựa trên việc thực hiện phép toán XOR mã số sinh viên của cả 3 thành viên trong nhóm:

$$\text{SEED} = 202416124 \oplus 202400076 \oplus 2416167 = 203875323$$

Cài đặt cụ thể trong file `src/core/position/zobrist_table.rs`:

```
1 /// Holds Zobrist random numbers used for fast, incremental position
2 /// hashing. Positional hashes are updated via XOR operations during
3 /// do_move/undo_move, entirely avoiding full-board hash recalculations.
4 pub(super) struct ZobristTable {
5     /// Random keys for every piece type on every one of the 90 squares:
6     /// Categorized as `pieces[piece_index][square_index]`.
7     pub pieces: [[u64; Square::COUNT]; Piece::COUNT],
8     /// XOR'ed into the hash if it is Black's turn to move.
9     pub side: u64,
10 }
11
12 pub(super) static ZOBRIST: ZobristTable = {
13     const SEED: u64 = 202416124 ^ 202400076 ^ 2416167;
14     let mut pieces = [[0u64; Square::COUNT]; Piece::COUNT];
15     let mut piece_idx = 0;
16
17     const fn gen_next(mut value: u64) -> u64 {
18         value = value
19             .wrapping_mul(6364136223846793005)
20             .wrapping_add(1442695040888963407);
21         value
22     }
23
24     let mut value = SEED;
25     while piece_idx < Piece::COUNT {
26         let mut square_idx = 0;
27         while square_idx < Square::COUNT {
28             value = gen_next(value);
29             pieces[piece_idx][square_idx] = value;
30             square_idx += 1;
31         }
32         piece_idx += 1;
33     }
34     let side = gen_next(value);
35     ZobristTable { pieces, side }
36 };
```

Listing 3.5: Khởi tạo Zobrist Table tĩnh tại thời điểm biên dịch

3.4.3 Cập nhật vi phân trạng thái $O(1)$

Khi di chuyển quân cờ, mã băm Zobrist không cần phải quét duyệt lại toàn bộ bàn cờ. Nhờ tính chất của phép XOR ($A \oplus A = 0$ và $0 \oplus B = B$), ta có thể cập nhật vi phân mã băm chỉ bằng các phép XOR tại các vị trí thay đổi.

Ví dụ, khi thực hiện một nước đi dịch chuyển một quân `piece` từ ô `from` đến ô `to`, mã băm thay đổi như sau:

```

1 // Remove piece from the starting square
2 hash ^= ZOBRIST.pieces[piece][from];
3 // Put the piece at the ending square
4 hash ^= ZOBRIST.pieces[piece][to];
5 // If there exist a piece at the ending square, remove it from the board
6 if let Some(captured) = captured_piece {
7     hash ^= ZOBRIST.pieces[captured][to];
8 }
9 // Change the side to move
10 hash ^= ZOBRIST.side;

```

Listing 3.6: Cập nhật vi phân Zobrist trong `do_move`

Cơ chế cập nhật vi phân này biến thao tác tính toán mã băm từ độ phức tạp $O(N)$ (với $N = 90$ ô cờ) về độ phức tạp $O(1)$ chỉ với 3 đến 4 phép XOR cơ bản, tiết kiệm tối đa tài nguyên CPU.

3.5 Bộ phân xử luật cờ tướng và phát hiện lặp trạng thái

Luật chơi cờ tướng Việt Nam và quốc tế có điểm khác biệt rất lớn so với cờ vua trong việc xử lý các trạng thái lặp lại thể trận. Trong cờ vua, nếu một thể trận lặp lại 3 lần (Threefold Repetition), ván đấu lập tức được xử hòa. Tuy nhiên, trong cờ tướng, luật chơi nghiêm cấm các hành vi mang tính chất cưỡng đoạt hoặc quấy rối vô hạn:

- **Chiếu dai (Perpetual Check):** Một bên liên tục chiếu tướng đối phương mà không thay đổi phương án tấn công. Bên chiếu dai bắt buộc phải thay đổi nước đi, nếu không sẽ bị xử thua.
- **Đuổi dai (Perpetual Chase):** Một bên liên tục sử dụng các quân tấn công (Xe, Mã, Pháo, Tốt đã qua sông) để uy hiếp, bắt một quân cờ của đối phương một cách lặp đi lặp lại. Bên đuổi dai cũng bị cấm và sẽ bị xử thua nếu không đổi nước đi.
- **Lặp nước bình thường:** Các tình huống hai bên luân phiên đổi nước đi tự vệ, hoặc quấy rối lẫn nhau (không vi phạm chiếu dai hay đuổi dai) sẽ được xử hòa.

Lngine tích hợp bộ trọng tài luật trong tệp [src/core/position/rule_judge.rs](#). Để phát hiện lặp lại thể trận, hệ thống lưu trữ toàn bộ lịch sử các mã băm Zobrist trong danh sách `history`. Khi tìm kiếm, engine sẽ duyệt ngược lịch sử để tìm các mã băm trùng lặp. Khi phát hiện trùng lặp thể trận ở nửa nước đi thứ i , bộ trọng tài sẽ kích hoạt giải thuật kiểm tra đuổi bắt (`chased`) để xác định hành vi của bên đi.

Giải thuật kiểm tra đuổi bắt sẽ thực hiện di chuyển thử nghiệm ảo trên bàn cờ. Nếu quân tấn công có giá trị thấp hơn hoặc bằng quân bị tấn công, hoặc quân bị tấn công đang đứng ở vị trí không được bảo vệ, nước đi đó sẽ được phân loại là hành vi “Đuổi” (`Chase`). Nếu một bên thực hiện liên tiếp các nước đi chiếu hoặc đuổi lặp lại trong chu kỳ thể trận, bộ trọng tài sẽ trả về điểm số phạt thua đối với bên vi phạm:

- Nếu bên đi phạm luật chiếu dai hoặc đuổi dai: Trả về điểm số bị chiếu bí sau d nước đi (`mated_in(ply)`).
- Nếu đối thủ phạm luật: Trả về điểm số chiếu bí đối thủ (`mate_in(ply)`).
- Nếu lặp nước hòa hợp lệ: Trả về điểm số hòa cờ (`score::ZERO`).

3.6 Thiết kế bảng chuyển vị Transposition Table

Bảng chuyển vị (Transposition Table) là bộ nhớ đệm cực kỳ quan trọng giúp ghi nhớ kết quả đánh giá và nước đi tốt nhất tại các nút đã duyệt. Do cây tìm kiếm cờ tướng có rất nhiều đường dẫn trùng lặp (hoán vị nước đi, ví dụ đi Xe trước rồi đi Mã sau so với đi Mã trước Xe sau), bảng chuyển vị giúp cắt tỉa các nhánh cờ trùng lặp này ngay tại gốc.

3.6.1 Cấu trúc dữ liệu tối ưu hóa bộ nhớ đệm

Để tối ưu hóa việc truy xuất bộ nhớ, cấu trúc lưu trữ của bảng băm `Storage` được thiết kế với kích thước cực kỳ nhỏ gọn, vừa vặn đúng 16 byte (128 bit). Điều này giúp các entry nằm thẳng hàng trong các đường truyền cache line của CPU, giảm thiểu tối đa hiện tượng cache miss:

```

1 #[derive(Clone, Debug)]
2 struct Storage {
3     /// Zobrist board hash key.
4     pub key: Key, // 8 bytes
5     /// The evaluation score (may be relative to mate).
6     pub score: Score, // 4 bytes
7     /// The best move found at this position.
8     pub best_move: Option<Move>, // 2 bytes
9     /// The type of bound and age this entry represents.
10    /// The lower 2 bits encode the bound type.
11    pub flags: Flags, // 1 byte
12    /// The search depth this score was evaluated to.
13    /// Negative depth is Quiescence search, non-negative is regular
14    /// search depth.
15    pub depth: i8, // 1 byte
16 }
```

Listing 3.7: Cấu trúc Storage 16-byte của bảng băm TT

Nhờ sử dụng kiểu dữ liệu nguyên tử `NonZeroU8` cho `flags`, trình biên dịch Rust thực hiện tối ưu hóa ngách (niche optimization) để nén toàn bộ thực thể `Option<Storage>` vào đúng 16 byte mà không cần thêm byte padding nào.

3.6.2 Chính sách thay thế của bảng chuyển vị

Khi xảy ra va chạm chỉ số bảng băm (hai thế cờ khác nhau ánh xạ vào cùng một chỉ số trong bảng), Lngine áp dụng chính sách thay thế kết hợp **Độ sâu** (Depth-preferred) và **Tuổi** (Age-preferred):

- **Độ sâu:** Các kết quả tìm kiếm ở độ sâu lớn hơn chứa nhiều thông tin chiến thuật hơn, do đó được ưu tiên giữ lại.
- **Tuổi (Age):** Mỗi lần thực hiện tìm kiếm mới ở nước đi tiếp theo, chỉ số tuổi của bảng băm (age) tăng lên. Các entry quá cũ (từ các nước đi trước đó) sẽ giảm giá trị tham chiếu và dễ bị ghi đè hơn.

Công thức kiểm tra điều kiện ghi đè entry trong `src\search\transposition_table.rs` được cài đặt như sau:

```

1 /// Stores search details into the Transposition Table using a
2 /// Depth-Preferred and Age-Preferred replacement strategy.
3 #[inline]
```

```

4 pub fn store(&mut self, key: Key, ply: u8, value: Entry) {
5     if self.table.is_empty() {
6         return;
7     }
8     let index = (key as usize) & self.size_mask;
9
10    if let Some(existing) = self.table[index].as_ref() {
11        let is_better =
12            value.depth >= existing.depth - self.relative_age(existing.
13            flags.age()) as i8;
14        if !is_better {
15            return; // Reject the new entry if it's not better than the
16            existing one
17        }
18        self.table[index] = Some(Storage {
19            key,
20            score: score::ply_independent(value.score, ply),
21            best_move: value.best_move,
22            flags: Flags {
23                data: unsafe { NonZeroU8::new_unchecked(value.bound as u8) | (
24                self.age << 2) },
25            },
26            depth: value.depth,
27        });
28    }

```

3.6.3 Chuyển đổi điểm số Mate phụ thuộc độ sâu (Mate Distance Pruning)

Một vấn đề kỹ thuật phức tạp khi sử dụng bảng băm là việc lưu trữ điểm số chiếu hết (Mate score). Điểm số chiếu hết trả về từ bộ tìm kiếm luôn phụ thuộc vào số nước đi từ gốc tới vị trí đó (*ply*). Nếu ta lưu trực tiếp điểm số này vào bảng chuyển vị, khi một nhánh tìm kiếm khác ở độ sâu khác truy vấn trùng entry này, điểm số chiếu hết sẽ bị lệch nước đi.

Lengine giải quyết triệt để bằng cách chuẩn hóa điểm số trước khi lưu và khôi phục khi truy vấn:

- **Khi lưu (Store):** Chuyển đổi điểm số từ phụ thuộc ply sang độc lập ply:

$$\text{Score}_{\text{TT}} = \begin{cases} \text{Score} + \text{ply} & (\text{nếu thắng chiếu hết}) \\ \text{Score} - \text{ply} & (\text{nếu thua chiếu hết}) \end{cases}$$

- **Khi truy vấn (Probe):** Khôi phục lại điểm số tương thích với ply hiện tại:

$$\text{Score}_{\text{Search}} = \begin{cases} \text{Score}_{\text{TT}} - \text{ply} & (\text{nếu thắng chiếu hết}) \\ \text{Score}_{\text{TT}} + \text{ply} & (\text{nếu thua chiếu hết}) \end{cases}$$

Chương 4

Mô hình tác tử và giải thuật

4.1 Mô hình chuyển đổi và Biểu diễn dữ liệu

4.1.1 Bitboards – Nền tảng tốc độ

Để một tác tử AI chơi cờ tướng có thể đạt tới độ sâu chiến thuật hàng chục nước đi trong thời gian tính bằng giây, bộ sinh nước đi (Move Generator) phải đạt tốc độ tối đa. Lengine giải quyết vấn đề này bằng cách biểu diễn bàn cờ thông qua hệ thống Bitboard 128-bit. Thay vì sử dụng các cấu trúc mảng lồng nhau gây tốn kém thời gian truy cập bộ nhớ, các bitboard cho phép thực thi các phép toán logic song song trên toàn bộ bàn cờ.

4.1.2 Cơ chế sinh nước đi bằng Magic Bitboards

Trong Cờ tướng, Tượng và Mã là hai loại quân có cơ chế di chuyển phức tạp vì chúng có thể bị cản (bị chặn) bởi các quân khác trên bàn cờ:

- **Tượng (Elephant):** Đi chéo 2 ô. Quân Tượng sẽ bị cản nếu ô ở giữa (mắt tượng) có quân đứng. Ngoài ra, Tượng bị giới hạn không được phép qua sông.
- **Mã (Horse):** Đi thẳng 1 ô rồi chéo 1 ô (tạo thành hình chữ nhật 2×1). Quân Mã bị cản nếu ô thẳng kế bên (chân mã) có quân chặn.
- **Mã ngược (KnightTo):** Một khái niệm ảo được sử dụng trong engine để tìm ngược xem có quân Mã nào của đối phương đang tấn công ô cờ hiện tại hay không, hỗ trợ phát hiện chiếu tướng cực kỳ nhanh chóng.

Ví dụ:

Hãy tưởng tượng quân Mã của bạn đang đứng ở giữa bàn cờ. Nó có tối đa 8 nước đi tiềm năng. Tuy nhiên, 8 nước đi này phụ thuộc hoàn toàn vào 4 ô “chân mã” xung quanh nó (trên, dưới, trái, phải). Mỗi ô chân mã có thể ở trạng thái trống (0) hoặc đang bị chiếm đóng (1).

Như vậy, đối với quân Mã, ta có $2^4 = 16$ trạng thái cản quân khác nhau. Thay vì viết 16 trường hợp **if-else** hoặc dùng vòng lặp sẽ làm CPU chậm lại do hiện tượng dự đoán sai nhánh (branch misprediction), chúng ta cần một cách tiếp cận toán học để lấy ra nước đi ngay lập tức.

Để sinh nước đi nhanh cho Tượng và Mã mà không cần rẽ nhánh, Lengine áp dụng kỹ thuật **Magic Bitboards 128-bit**. Kỹ thuật này hoạt động như một hàm băm hoàn hảo: nó gom trạng thái của các ô cản trên bàn cờ và “biến hóa” chúng thành một chỉ số duy nhất để tra cứu trực tiếp từ một mảng đã tính sẵn.

Công thức toán học cốt lõi:

$$\text{Index} = \frac{(\text{Occupied} \& \text{Mask}) \times \text{Magic}}{2^{128-\text{bits}}}$$

Trong đó:

- **Occupied:** Bitboard 128-bit chứa trạng thái hiện tại của toàn bộ bàn cờ (ô có quân là 1, ô trống là 0).
- **Mask (Mặt nạ):** Bitboard dùng để “lọc” ra vị trí các ô cần khả dĩ. Ví dụ, với quân Mã đang xét, mặt nạ sẽ che đi toàn bộ bàn cờ và chỉ giữ lại đúng 4 bit tại 4 ô chân mã.
- **Magic (Số ma thuật):** Một số 128-bit được hệ thống tìm kiếm từ trước. Phép nhân với số ma thuật này có tác dụng đẩy các bit đang phân tán về một cụm trên cùng của số 128-bit mà không gây ra va chạm (trùng lặp chỉ số).
- **bits:** Số lượng bit cần thiết để tạo mảng tra cứu. Với Mã có 4 ô cần, ta dùng 4 bit, tương ứng kích thước mảng là $2^4 = 16$ phần tử.

Sơ đồ dưới đây minh họa cách phép nhân ma thuật gom 4 bit chân mã (A, B, C, D) đang nằm rải rác trên bàn cờ 9×10 (90 bit) lên vị trí cao nhất của thanh ghi 128-bit, trước khi dịch phải để tạo thành mảng chỉ số từ 0 đến 15:

Occupied & Mask <i>Quân Mã (10 × 9)</i>	Số ma thuật 128-bit	Chỉ số ánh xạ <i>4 bit cao nhất chứa [A B C D]</i>
.....		[A B C D]
.....		
.....		
.....		
..... A	... một số. . .	
. . . D x C	. bit ma thuật. . .	. . . bit rác . .
..... B		
.....		
.....		
.....		

Bảng 4.1: Quá trình ánh xạ bit bằng phép nhân Magic Bitboard cho quân Mã

Cấu trúc và Thực thi trong Lngine

Cấu trúc tra bảng Magic được định nghĩa gọn gàng trong tệp [src/core/movegen/table s/types.rs](#):

```

1 /// One entry in the magic lookup table for a leaper piece on a single
  square.
2 #[derive(Clone, Copy)]
3 pub(in crate::core::movegen) struct Magic<const SIZE: usize> {
4     /// Bitmask of squares that affect this piece's mobility from this
    square
5     /// (blocking squares only - not the piece's own square or destinations
    ).
6     pub mask: u128,
```



```

7  /// Magic multiplier chosen so that `(occ & mask) * magic >> SHIFT`
    maps
8  /// each distinct subset of `mask` to a unique index into `attacks`.
9  pub magic: u128,
10 /// Precomputed attack bitboard for each possible occupancy subset.
11 pub attacks: [Bitboard; SIZE],
12 }
13
14 impl<const SIZE: usize> Magic<SIZE> {
15     /// Number of bits to shift after multiplication to obtain the table
    index.
16     /// `SIZE` must be a power of two; `SHIFT = 128 - log2(SIZE)`.
17     const SHIFT: u32 = 128 - SIZE.trailing_zeros();
18
19     /// Look up the attack bitboard for the given board occupancy.
20     #[inline]
21     pub const fn attack(&self, occupied: Bitboard) -> Bitboard {
22         // 1. Keep only the squares relevant to this piece's mobility.
23         // 2. Multiply by the magic to scatter those bits into the high
    bits.
24         // 3. Shift to a small index and return the precomputed attack set.
25         let idx = (occupied.raw() & self.mask).wrapping_mul(self.magic) >>
    Self::SHIFT;
26         self.attacks[idx as usize]
27     }
28 }

```

Listing 4.1: Cấu trúc Magic Bitboard trong Lengine

Nhờ cơ chế này, thay vì phải chạy vòng lặp hay kiểm tra điều kiện phức tạp, việc truy vấn toàn bộ các nước đi khả dĩ của một quân Tượng hoặc Mã giờ đây được thu gọn lại thành **một phép nhân nguyên 128-bit và một phép dịch bit**.

Ví dụ, để lấy nước đi của Tượng và loại bỏ các ô có quân nhà (`us_pieces`), đoạn mã sẽ vô cùng ngắn gọn và tối ưu hiệu suất:

```

1 let attacks = BISHOP_MAGICS[from_sq].attack(occupied) & !us_pieces;

```

4.1.3 Sinh nước đi cho Xe và Pháo

Quân Xe và Pháo là hai quân cờ có tầm hoạt động rộng, di chuyển theo đường thẳng (dọc hoặc ngang). Tuy nhiên, cơ chế ăn quân của chúng có sự khác biệt cốt lõi, làm cho việc sinh nước đi cho Pháo phức tạp hơn nhiều so với Xe:

- **Quân Xe:** Di chuyển và ăn quân dọc theo đường thẳng, bị chặn lại ngay tại quân cờ đầu tiên gặp phải trên đường đi.
- **Quân Pháo:** Di chuyển đến các ô trống giống hệt như Xe, nhưng để ăn quân của đối phương, Pháo bắt buộc phải nhảy qua đúng một quân cờ khác (được gọi là “ngòi pháo”).

Để tối ưu hóa tốc độ tính toán và tránh việc phải dùng các vòng lặp kiểm tra từng ô một (*ray-casting*) vốn tốn nhiều tài nguyên, Lengine áp dụng kỹ thuật **tra cứu bảng tĩnh**. Bàn cờ tướng tiêu chuẩn có kích thước 9 cột dọc (*File*) và 10 hàng ngang (*Rank*). Dựa vào đặc điểm này, engine số hóa toàn bộ trạng thái chiếm đóng của các đường đi thành các chuỗi bit:

- Một hàng ngang (9 ô) được biểu diễn trực tiếp bằng một số nguyên 9-bit.
- Một cột dọc (10 ô) được nén thành một số nguyên 10-bit.

Trích xuất trạng thái cột dọc

Trong biểu diễn bàn cờ bằng mảng bit một chiều (sử dụng số nguyên lớn như 128-bit), các ô trên cùng một cột dọc không nằm liền kề nhau mà nằm cách nhau 9 bit. Để có thể dùng trạng thái của cột làm chỉ số cho việc tra cứu, hệ thống cần gom 10 bit nằm cách quãng này lại thành một chuỗi 10-bit liền kề.

Hàm `gather_file_bits` thực hiện nhiệm vụ nén dữ liệu này bằng kỹ thuật nhân ma thuật:

```
1 /// Packers the 10 file bits (every 9th bit starting at `f`) into a
  contiguous
2 /// 10-bit integer. Splits across two u64 halves to stay within 64-bit
3 /// multiply range, using a magic multiplier to compress 5 spaced bits each
  .
4 #[inline]
5 const fn gather_file_bits(bits: u128, f: usize) -> usize {
6     let occ = bits >> f;
7     const STEP_MASK: u64 = 0x10_0804_0201;
8     const MAGIC_MULTIPLIER: u64 = 0x1010101010;
9     let low = (occ as u64 & STEP_MASK).wrapping_mul(MAGIC_MULTIPLIER) >>
    36;
10    let high = ((occ >> 45) as u64 & STEP_MASK).wrapping_mul(
    MAGIC_MULTIPLIER) >> 36;
11    ((low & 0x1F) | ((high & 0x1F) << 5)) as usize
12 }
```

Listing 4.2: Hàm trích xuất trạng thái cột dọc trong Lngine

Phân tích chi tiết thuật toán nén:

1. **Dịch bit:** Lệnh `bits >> f` dịch toàn bộ mảng bit của bàn cờ để đưa bit thấp nhất của cột `f` về vị trí số 0.
2. **Chia nhỏ dữ liệu để xử lý:** Vì khoảng cách giữa bit đầu và bit cuối của một cột là 90 bit (vượt quá dung lượng của một số nguyên 64-bit thông thường), Lngine tách quá trình nén thành hai nửa: nửa thấp (`low`) xử lý 5 ô dưới và nửa cao (`high`) xử lý 5 ô trên.
3. **Lọc và Nhân Ma Thuật:**
 - `STEP_MASK` đóng vai trò như một bộ lọc, chỉ giữ lại đúng các bit thuộc cột hiện tại và xóa bỏ mọi bit của các cột khác.
 - `MAGIC_MULTIPLIER` là một hằng số được tính toán kỹ lưỡng. Khi nhân giá trị đã lọc với hằng số này, các bit rời rạc sẽ bị dịch chuyển và dồn lên nhau sao cho chúng tập trung thành một cụm liền kề ở các bit cao nhất của kết quả.
4. **Dịch ngược và Gộp:** Kết quả được dịch lùi lại (`>> 36`) và sử dụng toán tử OR bitwise (`|`) để ghép 5 bit của nửa thấp với 5 bit của nửa cao. Kết quả cuối cùng trả về một số nguyên 10-bit hoàn chỉnh (có giá trị từ 0 đến 1023).

Tra cứu đường đi siêu tốc

Sau khi trích xuất thành công số nguyên 9-bit (cho hàng ngang) hoặc 10-bit (cho cột dọc) đại diện cho vị trí các quân cờ cản đường, số nguyên này sẽ được sử dụng trực tiếp làm chỉ mục. Engine chỉ cần truy cập vào hai mảng tĩnh đã được khởi tạo sẵn từ trước:

- `RANK_TABLE[trạng_thái_9_bit][vị_trí_hiện_tại]`
- `FILE_TABLE[trạng_thái_10_bit][vị_trí_hiện_tại]`

Hai bảng này chứa tất cả các kịch bản di chuyển hợp lệ của Xe và Pháo (bao gồm cả nước đi trống và nước ăn quân) tương ứng với mọi cách sắp xếp quân cản có thể xảy ra. Nhờ cơ chế này, Lngine loại bỏ hoàn toàn các vòng lặp dò tìm từng ô truyền thống. Độ phức tạp tính toán của mỗi nước đi trượt được giảm xuống chỉ còn $O(1)$, giúp tăng tốc độ sinh nước đi lên gấp nhiều lần.

4.2 Hàm đánh giá thế cờ

Hàm đánh giá chịu trách nhiệm lượng hóa ưu thế của thế cờ hiện tại từ góc nhìn của bên Đỏ dưới dạng một số nguyên:

4.2.1 Công thức tổng quát

$$\text{Score} = \text{TaperedScore}(\text{BaseScore} + \text{MobilityScore} + \text{DefenderScore}, \text{Phase})$$

Trong đó:

- **BaseScore** là tổng chênh lệch điểm số quân cờ cơ bản (Material) và vị trí (Piece-Square Tables) của hai bên.
- **MobilityScore** thưởng điểm cho bên có các quân Xe, Pháo, Mã hoạt động tự do và cơ động cao.
- **DefenderScore** thưởng điểm cho hệ thống Sĩ, Tượng che chở bảo vệ Tướng và bảo vệ lẫn nhau.
- **TaperedScore** thực hiện pha trộn động điểm số Trung cuộc và Tàn cuộc.

4.2.2 Nội suy pha ván cờ (Tapered Evaluation)

Để tránh sự thay đổi điểm số đột ngột khi chuyển giao giai đoạn (ví dụ quân cờ bị ăn dẫn đến thay đổi thuật toán đánh giá), Lngine áp dụng kỹ thuật nội suy tuyến tính dựa trên giá trị pha ván cờ (Game Phase):

$$\text{PhaseWeight} = \sum \text{PiecePhaseWeight}(pt)$$

Với quy ước trọng số pha: Xe = 2, Pháo = 2, Mã = 2, Sĩ = 1, Tượng = 1, Tốt/Tướng = 0. Tổng trọng số pha tối đa của một bàn cờ đầy đủ là `MAX_PHASE` = 32. Điểm số cuối cùng được tính bằng:

$$\text{Score} = \frac{\text{Score}_{\text{MG}} \times \text{Phase} + \text{Score}_{\text{EG}} \times (\text{MAX_PHASE} - \text{Phase})}{\text{MAX_PHASE}}$$

4.2.3 Tinh chỉnh tự động bằng Texel Tuning

Toàn bộ hệ thống trọng số tĩnh của Lngine, bao gồm giá trị cốt lõi của các quân cờ, hàng trăm chỉ số nội suy trong bảng vị trí, điểm thưởng cơ động và hệ số phòng thủ, không được thiết lập thủ công bằng tay theo cảm tính. Thay vào đó, chúng được huấn luyện tự động thông qua giải thuật tối ưu hóa hồi quy logistic toán học mang tên **Texel Tuning**. Phương pháp này biến bài toán tinh chỉnh cơ chế đánh giá cờ thành một bài toán học máy giám sát (*Supervised Learning*), nơi hệ thống tự học từ hàng triệu thế cờ thực tế.

Cơ chế sinh và xử lý dữ liệu huấn luyện

Hiệu năng và độ chính xác của thuật toán Texel Tuning phụ thuộc vào chất lượng của tập dữ liệu huấn luyện. Để xây dựng một tập dữ liệu khổng lồ vừa đảm bảo tính chuẩn mực chiến thuật, vừa bao phủ được các không gian trạng thái đa dạng, công cụ sinh dữ liệu của Lngine tổng hợp từ hai nguồn cốt lõi:

- **Biên bản ván đấu của các Đại sư (Masters Games):** Nguồn dữ liệu này được trích xuất từ các kỳ đài, giải đấu chuyên nghiệp đỉnh cao của con người. Ưu điểm của dữ liệu đại sư là cung cấp các cấu trúc hệ thống khai cuộc cực kỳ bài bản, các hình thế trung cuộc phức tạp và chiến lược điều quân có chiều sâu tuệ giác cao.
- **Dữ liệu tự đấu từ Fairy-Stockfish (Fairy-Stockfish Games):** Vì dữ liệu của con người có giới hạn và thường thiếu các thế cờ tàn cuộc hoặc các sai lầm ngẫu nhiên, Lngine sử dụng engine *Fairy-Stockfish* (biến thể tối ưu của Stockfish dành cho cờ tướng) để chạy chế độ tự đấu. Quá trình này áp dụng kỹ thuật xáo trộn ngẫu nhiên một số nước đi đầu kết hợp với việc giới hạn độ sâu tính toán để kích thích sinh ra hàng triệu hình thế độc lạ, mô phỏng lại toàn bộ các kịch bản sơ hở có thể xảy ra trên bàn cờ.

Quy trình tiền xử lý và lọc nhiễu:

Sau khi thu thập, tập dữ liệu thô phải đi qua một bộ lọc nghiêm ngặt trước khi tinh chỉnh:

1. **Loại bỏ thế cờ chiến thuật quá mức:** Thuật toán loại bỏ toàn bộ các thế cờ đang bị chiếu tướng hoặc các vị trí có nước bắt quân trực tiếp. Điều này đảm bảo Texel Tuning chỉ tập trung tối ưu hóa hàm đánh giá tĩnh thay vì bị nhiễu bởi các đòn phối hợp tính toán sâu.
2. **Gán nhãn đồng bộ góc nhìn:** Kết quả chung cuộc của ván đấu được chuyển đổi chính xác về góc nhìn của bên chuẩn bị thực hiện nước đi tại thế cờ đó. Nếu bên đi trước giành chiến thắng cuối cùng, giá trị R_i được gán là 1.0, hòa là 0.5, và thất bại là 0.0, tạo tiền đề cho hàm kích hoạt Sigmoid hội tụ chính xác tuyệt đối.

Tổng cộng, với 40 711 ván đấu từ các Đại sư, và 100 000 ván đấu từ Fairy-Stockfish, bộ dữ liệu cuối cùng có được 4 783 928 thế cờ với đầy đủ label.

Cơ sở toán học và Hàm mất mát

Mục tiêu cốt lõi của Texel Tuning là tìm ra một bộ tham số tối ưu $\vec{\theta}$ chứa cả hai thành phần Trung cuộc (*Middle Game* - *MG*) và Tàn cuộc (*End Game* - *EG*) sao cho hàm sai

số bình phương trung bình (*Mean Squared Error - MSE*) trên tập dữ liệu gồm N trạng thái bàn cờ đạt giá trị nhỏ nhất:

$$\text{MSE}(\vec{\theta}) = \frac{1}{N} \sum_{i=1}^N \left(R_i - \sigma \left(E(S_i, \vec{\theta}) \right) \right)^2 \quad (4.1)$$

Trong đó:

- $R_i \in \{1.0, 0.5, 0.0\}$ là kết quả thực tế của ván đấu chứa thế cờ thứ i (tương ứng với các trạng thái Thắng, Hòa, Thua dưới góc nhìn của bên đi trước).
- $E(S_i, \vec{\theta})$ là hàm đánh giá tĩnh thế cờ S_i bằng bộ tham số $\vec{\theta}$. Giá trị này được tính toán dựa trên kỹ thuật nội suy theo pha bàn cờ (*Tapered Evaluation*):

$$E(S_i, \vec{\theta}) = \frac{\text{Score}_{\text{MG}} \times \text{Phase} + \text{Score}_{\text{EG}} \times (32 - \text{Phase})}{32} \quad (4.2)$$

- $\sigma(x)$ là hàm kích hoạt Sigmoid có nhiệm vụ ánh xạ điểm số cờ thuần túy (vốn là một số nguyên không giới hạn) sang một không gian xác suất $[0, 1]$ đại diện cho cơ hội chiến thắng dự kiến:

$$\sigma(x) = \frac{1}{1 + 10^{-\frac{K \cdot x}{400}}} \quad (4.3)$$

Hệ số tỷ lệ K đóng vai trò co giãn đồ thị hàm số nhằm đảm bảo mối tương quan tuyến tính giữa điểm số của engine và tỷ lệ thắng thực tế. Trong Lngine, K không phải là một hằng số cố định mà được tối ưu hóa động ở mỗi chu kỳ huấn luyện bằng cách sử dụng thuật toán tìm kiếm tam phân (*Ternary Search*) để tìm kiếm nhanh chóng vùng chứa sai số thấp nhất.

Thực tế triển khai và Kết quả huấn luyện

Trong thực tế triển khai, công cụ huấn luyện của Lngine đã tối ưu hóa toàn diện bài toán hiệu năng thông qua việc kết hợp cấu trúc dữ liệu và xử lý song song. Nhờ việc trích xuất trạng thái bàn cờ dưới dạng **Đặc trưng thưa (*Sparse Features*)**, bài toán tính toán ma trận khổng lồ được thu gọn đáng kể. Dựa trên không gian đặc trưng này, bộ **tối ưu hóa Adam** có thể thực hiện tính toán đạo hàm riêng và cập nhật đồng thời hàng ngàn trọng số tĩnh để tối thiểu hóa hàm mất mát.

Để tận dụng tối đa sức mạnh phần cứng, toàn bộ quá trình quét tập dữ liệu để tính toán sai số và tích lũy đạo hàm được phân bổ thực thi song song trên kiến trúc đa luồng của CPU. Cơ chế này cho phép hệ thống xử lý hàng triệu thế cờ trong mỗi vòng lặp và hội tụ cực kỳ nhanh chóng về điểm tối ưu toàn cục mà không gặp hiện tượng nghẽn cổ chai.

Về mặt chi phí tài nguyên tính toán, toàn bộ vòng đời huấn luyện được chia làm hai giai đoạn chính với khối lượng công việc cụ thể như sau:

- **Giai đoạn sinh dữ liệu:** Quá trình chạy tự động, trích xuất dữ liệu Đại sư, lọc nhiễu và gán nhãn hàng triệu thế cờ EPD tiêu tốn khoảng **28 giờ CPU**.
- **Giai đoạn tinh chỉnh:** Quá trình chạy thuật toán Adam để lặp và tối ưu hóa bộ trọng số mất khoảng **5 giờ CPU** thực thi liên tục cho đến khi thuật toán đạt ngưỡng hội tụ.

Hiệu quả của giải thuật được minh chứng bằng sự sụt giảm rõ rệt của hàm mất mát (MSE) và sự tự hiệu chỉnh của hệ số tỷ lệ K . Kết quả đo lường ghi nhận sự khác biệt lớn giữa bộ tham số khởi tạo (chưa tinh chỉnh) và bộ tham số hoàn thiện được tổng hợp chi tiết trong Bảng 4.2.

Trạng thái	Hệ số tỷ lệ K	Sai số toàn phương trung bình (MSE)
Khởi tạo	0.233339	0.0620215701
Hội tụ	0.549194	0.0550919271

Bảng 4.2: So sánh hệ số tỷ lệ K và sai số MSE trước và sau khi tinh chỉnh

Sự sụt giảm hơn 11% của chỉ số MSE trên một tập dữ liệu thử nghiệm khổng lồ là một bước nhảy vọt về độ chính xác.

Để kiểm chứng sức mạnh thực tế của bộ trọng số mới sau khi được tối ưu hóa bằng Texel Tuning, một giải đấu kiểm thử gồm 1000 ván đấu đã được tổ chức khép kín. Giải đấu diễn ra giữa phiên bản Lngine 1.8.0 (được trang bị bộ tham số vừa hội tụ) và phiên bản tiền nhiệm Lngine 1.7.0 (sử dụng bộ tham số gốc cấu hình thủ công).

Kết quả thực chiến cho thấy sự vượt trội hoàn toàn của hệ thống đánh giá mới. Chi tiết các chỉ số thống kê được trình bày trong Bảng 4.3, Lngine 1.8.0 đạt tỷ lệ thắng áp đảo với điểm số 79.7%. Đặc biệt, sức mạnh tổng thể của engine gia tăng tới **+237.6 Elo** so với phiên bản trước, đi kèm với Chỉ số tin cậy vượt trội (*Likelihood of Superiority - LOS*) đạt mức tuyệt đối 100.0%.

Thành quả này minh chứng rõ ràng cho sức mạnh của Machine Learning trong việc thay thế hoàn toàn các đánh giá cảm tính của con người, giúp engine cờ nâng tầm chiến thuật một cách đột phá.

Chỉ số đo lường	Lngine 1.8.0	Lngine 1.7.0
Thắng	719	125
Hòa	156	156
Thua	125	719
Tổng điểm	797.0/1000 (79.7%)	203.0/1000 (20.3%)
Chênh lệch Elo	+237.6	-237.6
Sai số chuẩn	± 11.9	—
Khoảng tin cậy 95%	[214.3, 260.9]	—
Khả năng vượt trội	100.0%	—
Tỷ lệ hòa	15.6%	—

Bảng 4.3: So sánh sức mạnh giữa Lngine 1.8.0 và Lngine 1.7.0 qua 1000 ván đấu

4.2.4 Công thức chi tiết từng thành phần đánh giá

Các tham số trọng số vật chất (Material Values)

Trong Lngine, giá trị cơ bản của các quân cờ không phải là một hằng số cố định mà được chia làm hai pha đánh giá riêng biệt để phản ánh đúng tầm quan trọng chiến thuật của chúng tại từng thời điểm: pha Trung cuộc (*Middlegame - MG*) khi bàn cờ còn đông quân, cấu trúc phức tạp; và pha Tàn cuộc (*Endgame - EG*) khi quân cờ thưa thớt, tập trung vào việc di chuyển Tướng và các ưu thế nhỏ như phong cấp Tốt.

Sau quá trình tối ưu hóa tự động bằng Texel Tuning, bộ trọng số vật chất cốt lõi của Lngine đã được máy học tái định chuẩn. Kết quả tinh chỉnh cho thấy sự thay đổi lớn về cách engine đánh giá sức mạnh quân cờ, chi tiết được trình bày trong Bảng 4.4.

Quân cờ (Piece Type)	Trung cuộc (MG)	Tàn cuộc (EG)
Xe	1344	611
Pháo	644	225
Mã	593	271
Sĩ	186	119
Tượng	179	108
Tốt chưa qua sông	85	87
Tốt đã qua sông	186	118

Bảng 4.4: Trọng số vật chất của các quân cờ trong Lngine sau Texel Tuning

Có thể thấy, thay vì thiết lập thủ công, thuật toán đã tự động gán điểm Trung cuộc cao hơn đáng kể so với Tàn cuộc cho các quân tấn công chủ lực (Xe, Pháo, Mã). Điều này phản ánh thực tế rằng sức mạnh phối hợp và khả năng tạo đột biến của các quân này trong giai đoạn đầu trận mang lại lợi thế quyết định hơn nhiều so với khi chúng bị đơn lẻ ở cuối ván cờ. Tốt sau khi qua sông (*Pawn Crossed*) cũng chứng kiến sức mạnh gia tăng đột biến, ngang ngửa với giá trị phòng thủ của một quân Sĩ.

Bảng vị trí quân cờ (Piece-Square Tables)

Bên cạnh giá trị vật chất cốt lõi, sức mạnh thực tế của một quân cờ còn phụ thuộc rất lớn vào tọa độ của nó trên bàn cờ. Ví dụ: Mã đứng ở tâm bàn cờ (Mã giao) sẽ có sức ảnh hưởng vượt trội so với Mã kẹt ở góc; Tượng nằm ở tuyến đáy an toàn hơn khi bị ép trời lên lầu 3. Để lượng hóa yếu tố không gian này, Lngine sử dụng **Bảng vị trí quân cờ**.

Mỗi loại quân cờ (Tượng, Sĩ, Tượng, Mã, Xe, Pháo, Tốt) đều sở hữu một bảng PST riêng biệt. Vì bàn cờ tướng tiêu chuẩn có kích thước 9×10 , mỗi bảng PST về mặt logic sẽ chứa 90 giá trị tương ứng với 90 ô tọa độ. Điểm PST của một phe được tính bằng tổng điểm vị trí của tất cả các quân cờ thuộc phe đó đang tồn tại trên bàn.

Để tối ưu hóa không gian lưu trữ và tăng tốc độ hội tụ cho thuật toán Texel Tuning, Lngine áp dụng hai kỹ thuật cốt lõi trong việc tổ chức PST:

1. **Tính đối xứng dọc:** Bàn cờ tướng có tính đối xứng gương qua trục dọc trung tâm (cột số 4 - cột Tượng). Một quân cờ đứng ở mép trái (cột 0) sẽ có giá trị chiến thuật tương đương khi đứng ở mép phải (cột 8). Do đó, thay vì phải tối ưu độc lập 90 tham số cho mỗi quân, hệ thống chỉ lưu trữ và tinh chỉnh $10 \times 5 = 50$ tham số độc lập (từ cột 0 đến cột 4). Các tọa độ nằm ở nửa phải bàn cờ sẽ được ánh xạ ngược lại nửa trái tương ứng khi truy xuất.
2. **Góc nhìn tương đối:** Các bảng PST được định nghĩa hoàn toàn dưới góc nhìn của phe Đỏ (quân đi từ dưới lên trên). Khi tính toán điểm cho phe Đen, tọa độ của quân Đen sẽ được lật ngược (*mirrored*) về mặt hàng ngang để sử dụng chung cấu trúc bảng với phe Đỏ, loại bỏ sự dư thừa dữ liệu.

Tương tự như giá trị vật chất, để phản ánh sự thay đổi tính chất chiến thuật qua các giai đoạn ván đấu, toàn bộ các bảng PST của Lngine đều là nội suy pha. Mỗi ô tọa độ

không chỉ chứa một con số duy nhất mà chứa một cấu trúc bao gồm một cặp điểm (*Trung cuộc*, *Tàn cuộc*).

Với 7 loại quân cờ và 50 thông số độc lập cho mỗi quân, hệ thống PST đóng góp tổng cộng $7 \times 50 \times 2 = 700$ trọng số vào hàm đánh giá. Toàn bộ 700 trọng số vị trí này đã được giải thuật Adam Optimizer liên tục tính toán đạo hàm và tự động điều chỉnh đến độ chính xác cao nhất thông qua hàng triệu thế cờ, giúp Lngine sở hữu khả năng cảm nhận vị trí sắc bén ngang tầm các đại sư con người.

Điểm cơ động (Mobility Score)

Điểm cơ động là đại lượng đánh giá mức độ kiểm soát không gian và khả năng triển khai linh hoạt của các quân chiến đấu chủ lực (Xe, Pháo, Mã). Về mặt chiến thuật, một quân cờ bị kẹt, thiếu không gian di chuyển sẽ bị hệ thống trừ điểm (phạt), ngược lại, một quân cờ hoạt động tự do, kiểm soát được nhiều giao điểm chiến lược sẽ nhận được điểm thưởng tương xứng.

Công thức tính chênh lệch điểm cơ động toàn cục giữa hai phe được biểu diễn như sau:

$$\text{MobilityScore} = \text{Mobility}_{\text{Red}} - \text{Mobility}_{\text{Black}} \quad (4.4)$$

Với điểm cơ động tổng hợp của từng phe được tính bằng cách cộng dồn điểm số của tất cả các quân Xe, Pháo, Mã đang hiện diện trên bàn cờ:

$$\begin{aligned} \text{Mobility}_{\text{Side}} = & \sum_{p \in \text{Knights}} M_{\text{Knight}}(\text{count}(p)) \\ & + \sum_{p \in \text{Rooks}} M_{\text{Rook}}(\text{count}(p)) \\ & + \sum_{p \in \text{Cannons}} M_{\text{Cannon}}(\text{count}(p)) \end{aligned}$$

Trong đó, $\text{count}(p)$ là hàm đếm số lượng nước đi khả dĩ (*pseudo-legal moves*) của quân cờ p tại thời điểm hiện tại, loại trừ các nước đi chồng lấp lên ô đang bị chiếm đóng bởi quân đồng minh.

Để loại bỏ chi phí tính toán các hàm đánh giá phi tuyến trong quá trình tìm kiếm sâu, Lngine thiết kế các hàm $M(\text{count})$ dưới dạng mảng tra cứu tĩnh một chiều. Mỗi phần tử trong mảng là một cấu trúc điểm gộp (*Trung cuộc*, *Tàn cuộc*) tương thích với cơ chế nội suy pha. Các mảng này là kết quả trực tiếp được trích xuất từ giải thuật Texel Tuning, phản ánh chính xác giá trị thực tế của từng mức độ tự do:

- **Mã** (M_{Knight} , từ 0 đến 8 nước): (38, -172), (52, -110), (63, -91), (79, -82), (89, -87), (98, -81), (105, -94), (117, -96), (128, -102).
- **Xe** (M_{Rook} , từ 0 đến 17 nước): (7, -57), (181, -180), (194, -237), (208, -229), (222, -227), (238, -220), (243, -215), (241, -180), (250, -182), (257, -182), (262, -189), (271, -198), (277, -198), (277, -195), (278, -218), (284, -242), (274, -235), (265, -235).
- **Pháo** (M_{Cannon} , từ 0 đến 17 nước): (42, -108), (68, -56), (69, -39), (86, -53), (92, -53), (103, -58), (107, -52), (101, -42), (104, -50), (108, -49), (109, -44), (114, -51), (122, -58), (125, -54), (123, -57), (142, -69), (154, -80), (164, -82).

Điểm thưởng hệ thống bảo vệ (Defender Score)

Trong cờ tướng, hệ thống Sĩ và Tượng đóng vai trò lá chắn phòng thủ tuyệt đối bảo vệ Tướng. Lngine đánh giá sức mạnh phòng thủ dựa trên số lượng Sĩ và Tượng đang hoạt động của mỗi bên.

Công thức tính điểm phòng vệ toàn cục:

$$\text{DefenderScore} = D_{\text{Red}} - D_{\text{Black}}$$

Với điểm phòng vệ của một bên được tính bằng:

$$D_{\text{Side}} = \text{Bonus}_{\text{Advisor}}(\text{count}(\text{Advisor})) + \text{Bonus}_{\text{Elephant}}(\text{count}(\text{Elephant}))$$

Trong đó, hàm $\text{count} \in \{0, 1, 2\}$ trả về số lượng quân phòng ngự tương ứng còn sót lại trên bàn cờ. Để phản ánh đúng tầm quan trọng của hệ thống phòng ngự qua từng giai đoạn, các hệ số Bonus được lưu trữ dưới dạng cặp điểm số (*Trung cuộc*, *Tàn cuộc*) phục vụ cho cơ chế nội suy pha (*Tapered Evaluation*):

- **Sĩ:** Hệ số lần lượt cho trường hợp còn 0 Sĩ, 1 Sĩ và đủ 2 Sĩ là: $(-32, 42)$, $(44, 20)$ và $(24, -19)$.
- **Tượng:** Hệ số lần lượt cho trường hợp còn 0 Tượng, 1 Tượng và đủ 2 Tượng là: $(-17, 48)$, $(13, 19)$ và $(45, -30)$.

Các giá trị thưởng/phạt có vẻ ngược trực giác (ví dụ: mất sạch Sĩ lại được cộng điểm *Tàn cuộc*) hoàn toàn không phải do thiết lập thủ công, mà là kết quả cân bằng phi tuyến tính tối ưu nhất do thuật toán Texel Tuning tự động tính toán ra dựa trên hàng triệu thế cờ.

4.3 Chiến lược tìm kiếm

Lngine triển khai cấu trúc tìm kiếm đối kháng dựa trên thuật toán cốt lõi Fail-Soft Alpha-Beta Negamax. Nhằm tối đa hóa độ sâu tìm kiếm trong giới hạn thời gian thực, engine kết hợp hàng loạt các kỹ thuật tự động mở rộng không gian duyệt tại các điểm nóng chiến thuật và cắt tỉa mạnh tay ở các nhánh kém hiệu quả.

4.3.1 Thuật toán Alpha-Beta và Phân loại nút mạng

Cốt lõi của tìm kiếm đối kháng trong Lngine là thuật toán **Negamax** kết hợp cắt tỉa **Alpha-Beta**. Negamax là một dạng rút gọn của thuật toán Minimax, tận dụng tính chất toán học của trò chơi zero-sum: tối đa hóa lợi thế của mình chính là tối thiểu hóa lợi thế của đối thủ ($\max(a, b) = -\min(-a, -b)$).

Để tối ưu hóa, thuật toán duy trì một “cửa sổ kỳ vọng” $[\alpha, \beta]$, trong đó:

- α (Alpha): Điểm số an toàn tối thiểu mà phe ta đã nắm chắc.
- β (Beta): Điểm số tối đa mà đối phương có thể cho phép phe ta đạt được.

Trong quá trình duyệt cây, nếu tại bất kỳ thời điểm nào điểm số hiện tại vượt qua ngưỡng β , ta biết chắc đối phương sẽ không bao giờ chọn nhánh này ở lượt đi trước của họ. Do đó, thuật toán có quyền ngừng duyệt ngay lập tức (cắt tỉa) nhánh đó.

Dựa trên sự tương tác của điểm số với cửa sổ $[\alpha, \beta]$, toàn bộ các trạng thái bàn cờ (nút mạng) trong cây tìm kiếm được Lngine phân loại thành ba nhóm cơ bản:

1. **Nút PV (PV-Node / Principal Variation Node):** Đây là các nút mà điểm số trả về lọt hẳn vào bên trong cửa sổ kỳ vọng ($\alpha < \text{score} < \beta$). Tại đây, thuật toán chưa thể chứng minh được nhánh này là vô giá trị hay quá tốt để đối thủ né tránh. Do đó, nút PV yêu cầu phải duyệt *toàn bộ* các nước đi con hợp lệ để tìm ra điểm số chính xác tuyệt đối. Số lượng nút PV trong cây chiếm tỷ lệ nhỏ nhưng tiêu tốn nhiều tài nguyên xử lý nhất.
2. **Nút Cắt tĩa (Cut-Node):** Đây là các nút mà chỉ cần duyệt một (hoặc một vài) nước đi đầu tiên đã tìm thấy điểm số lớn hơn hoặc bằng biên trên ($\text{score} \geq \beta$). Hiện tượng này gọi là **Fail-high** (hoặc Cắt tĩa Beta - *Beta Cutoff*). Về mặt chiến thuật mà nói, ta vừa tìm thấy một nước đi quá mạnh (ví dụ: ăn được Xe miễn phí), khiến đối phương thiệt hại nặng. Hệ quả tất yếu là ở lượt trước đó, đối phương khôn ngoan sẽ đi một nước khác để tránh việc mất Xe này. Do đó, ta không cần tốn thời gian tính tiếp các nước còn lại của nút này.
3. **Nút Toàn bộ (All-Node):** Đây là các nút mà sau khi bắt buộc phải duyệt *tất cả* các nước đi hợp lệ, không có bất kỳ một nước đi nào giúp điểm số vượt qua được mức an toàn α ($\text{score} \leq \alpha$). Hiện tượng này gọi là **Fail-low**. Về mặt chiến thuật mà nói, dù ta có vùng vẫy thế nào ở thế cờ này, mọi kết quả nhận được đều tồi tệ hơn một sự lựa chọn thay thế mà ta đã tìm thấy ở nhánh trước đó. Nút này được lưu vào lịch sử như một nước đi sai lầm cần né tránh.

Việc phân biệt rạch ròi ba loại nút này là cơ sở tối quan trọng. Mọi chiến thuật *Sắp xếp nước đi* và *Cắt tĩa* của Lngine ở các phần tiếp theo đều hướng đến một mục tiêu duy nhất: Tối đa hóa số lượng Cut-Node (để cắt tĩa thật sớm) và dùng các phép xấp xỉ để giảm thiểu tối đa độ sâu phải duyệt của các All-Node.

4.3.2 Quản lý thời gian

Trong các giải đấu cờ máy, việc quyết định khi nào nên dừng suy nghĩ quan trọng không kém việc engine tính toán như thế nào. Nếu dừng quá sớm, engine có thể đi những nước yếu do chưa nhìn đủ sâu; nếu nghĩ quá lâu, engine có thể bị xử thua vì hết thời gian. Mục tiêu của hệ thống quản lý thời gian là phân bổ quỹ thời gian còn lại của ván đấu một cách thông minh, ưu tiên đào sâu ở những thế cờ phức tạp nhưng vẫn đảm bảo tính an toàn tuyệt đối về mặt thời gian cho toàn bộ phần còn lại của trận đấu.

Để thực hiện điều này, Lngine chia ngân sách thời gian cho mỗi nước đi thành hai ngưỡng độc lập: Giới hạn mềm (*Soft-Bound*) và Giới hạn cứng (*Hard-Bound*), đồng thời luôn trừ hao một khoảng trễ an toàn $\text{TimeOverhead} = 15\text{ms}$ để bù đắp cho thời gian trễ khi giao tiếp qua giao thức UCI.

- **Phân bổ quỹ thời gian:** Tùy thuộc vào luật thời gian nhận được từ GUI, thuật toán sẽ tính toán hạn mức:
 - Nếu biết trước số nước đi cần hoàn thành: Thời gian được chia đều cho số nước đi đó, cộng với thời gian cộng thêm sau khi đi một nước.
 - Nếu không có giới hạn số nước đi cụ thể: Lngine sử dụng tham số phân bổ tĩnh. Gọi tổng quỹ thời gian cho một nước là $\text{Total} = \text{TimeLeft} + \text{Increment}$, hệ thống quy định Giới hạn mềm là $\text{Total}/40$ và Giới hạn cứng tối đa là $\text{Total}/8$.

- **Cơ chế Giới hạn mềm (Soft-Bound):** Giới hạn này được tích hợp trực tiếp vào vòng lặp tìm kiếm sâu dần (*Iterative Deepening*). Ngay sau khi hoàn thành trọn vẹn việc tìm kiếm ở một độ sâu d , thuật toán sẽ gọi hàm kiểm tra nếu quỹ thời gian đã vượt ngưỡng mềm, engine nhận định rằng việc khởi tạo tìm kiếm ở độ sâu $d + 1$ là quá rủi ro (vì độ sâu sau thường tốn thời gian theo cấp số nhân so với độ sâu trước). Khi đó, vòng lặp bị ngắt và engine lập tức chốt nước đi tốt nhất hiện tại để đảm bảo an toàn thời gian.
- **Cơ chế Giới hạn cứng (Hard-Bound):** Đây là “công tắc khẩn cấp”. Trong lúc engine đang mắc kẹt sâu bên trong một nhánh rẽ khổng lồ của cây Negamax, nếu phát hiện vượt thời gian tối đa cho phép, cờ tín hiệu `keep_running = false` sẽ được kích hoạt trên toàn hệ thống. Tính năng này ép toàn bộ các nút Alpha-Beta đang chạy dở dang phải thoái lui ngay lập tức, hủy bỏ nhánh đang xét để trả về kết quả an toàn trước khi hết giờ.

4.3.3 Sắp xếp nước đi

Hiệu năng cắt tỉa của thuật toán Alpha-Beta phụ thuộc hoàn toàn vào việc hệ thống có duyệt các nước đi tốt nhất trước hay không. Nếu nước đi tối ưu được xét đầu tiên, biên α sẽ được nâng lên nhanh chóng, tạo ra các pha cắt tỉa β sớm ở các nhánh khác.

Lngine gán điểm ưu tiên sắp xếp cho mỗi nước đi theo một trật tự được phân cấp nghiêm ngặt:

1. **Nước đi từ Bảng băm:** Nước đi tốt nhất được lưu trong Transposition Table từ các lần duyệt trước luôn được ưu tiên tuyệt đối ($\text{Score} = 2 \times 10^9$).
2. **Nước đi ăn quân:** Được sắp xếp theo heuristic MVV-LVA (*Most Valuable Victim - Least Valuable Attacker*) để ưu tiên các nước ăn quân dùng quân nhỏ ăn quân lớn. Công thức: $\text{Score} = 10^9 + \text{Victim} \times 10^7 - \text{Attacker} \times 10^5$.
3. **Nước đi sát thủ:** Các nước đi không ăn quân nhưng đã từng gây ra cắt tỉa β ở các nhánh khác cùng độ sâu ($\text{Score} = 9 \times 10^8$).
4. **Nước đi lịch sử:** Tích lũy điểm thưởng dựa trên lịch sử thành công của các nước đi không ăn quân trong quá khứ.

4.3.4 Tìm kiếm sâu dần (Iterative Deepening)

Thay vì ngay lập tức ra lệnh cho thuật toán tìm kiếm một mạch ở độ sâu tối đa N , engine sẽ duyệt lặp lại nhiều lần, bắt đầu từ độ sâu 1, sau đó là 2, 3... cho đến N . Quá trình này thoát nhìn có vẻ lãng phí tính toán, nhưng thực tế lại mang đến lợi ích khổng lồ: ở mỗi độ sâu d , thuật toán sẽ lưu lại nước đi tốt nhất vào Bảng băm (TT Move). Khi duyệt đến độ sâu $d + 1$, nước đi này sẽ được ưu tiên duyệt đầu tiên, giúp biên Alpha tăng vọt ngay từ sớm và tạo ra hiệu ứng cắt tỉa dây chuyền tối đa. Ngoài ra, nó giúp engine luôn có sẵn một nước đi tốt nhất để trả về trong trường hợp bị hết thời gian đột ngột.

Lngine sử dụng một vòng lặp `while` tăng dần độ sâu lên đến `max_depth`. Tại mỗi vòng lặp, engine cập nhật trung gian kết quả về giao diện người dùng thông qua giao thức UCI. Quan trọng nhất, engine liên tục kiểm tra giới hạn thời gian mềm. Nếu thuật toán nhận thấy không còn đủ thời gian an toàn để hoàn thành vòng lặp tiếp theo, nó sẽ tự động ngắt sớm và chốt nước đi tốt nhất của độ sâu vừa hoàn thành.

4.3.5 Cửa sổ kỳ vọng (Aspiration Windows & Fail-high Reductions)

Kỹ thuật này là sự kết hợp hoàn hảo với Iterative Deepening. Vì ta đã biết điểm số gần đúng của thế cờ từ kết quả của độ sâu $N - 1$, ta không cần thiết phải khởi tạo Alpha-Beta với một cửa sổ vô cực $[-\infty, +\infty]$ ở độ sâu N . Thay vào đó, ta chỉ mở một cửa sổ nhỏ xung quanh điểm số đó. Cửa sổ càng hẹp, số lượng nhánh bị cắt tỉa càng nhiều.

Thêm vào đó, Lngine đưa vào kỹ thuật *Fail-high Reductions*: Nếu một thế cờ liên tục phá vỡ biên trên của cửa sổ, điều đó chứng tỏ phe ta đang có một nước đi cực kỳ áp đảo. Khi đó, ta không cần phải tốn tài nguyên duyệt hết độ sâu tối đa chỉ để đo lường xem nước đi đó "tốt đến mức nào", mà có thể tự động giảm độ sâu duyệt xuống để chốt kết quả và phản hồi nhanh hơn.

Trong Lngine, cửa sổ kỳ vọng chỉ được kích hoạt từ độ sâu ≥ 6 . Bán kính ban đầu δ được đặt mặc định là 25 *centipawns* và được thu hẹp linh hoạt theo độ sâu: $\delta = \max(25 - \text{depth}, 10)$. Quá trình tìm kiếm lặp lại như sau:

- Nếu điểm trả về lọt vào trong cửa sổ ($\alpha < \text{score} < \beta$): Tìm kiếm thành công, trả về điểm số.
- Nếu $\text{score} \leq \alpha$ (*Fail-low*): Thế trận tiềm ẩn rủi ro lớn hơn dự kiến. Cửa sổ mở rộng phía dưới ($\alpha = \alpha - \delta$) và thuật toán bắt buộc phải duyệt lại với **độ sâu giữ nguyên** để tìm ra phương án phòng thủ.
- Nếu $\text{score} \geq \beta$ (*Fail-high*): Thế trận rất tốt. Cửa sổ mở rộng phía trên ($\beta = \beta + \delta$) và biến đếm `consecutive_fail_high` tăng thêm 1. Độ sâu duyệt lại được cắt giảm xuống thành: $(\text{depth} - \text{consecutive_fail_high}).\max(1)$.

Mỗi lần vượt rào thất bại, δ sẽ được nhân đôi ($\delta = \delta \times 2$) để cửa sổ mở rộng nhanh chóng cho lần tìm kiếm lại tiếp theo.

4.3.6 Tìm kiếm biến thể chính (Principal Variation Search - PVS)

Với sắp xếp nước đi tốt, ta có niềm tin rằng nước đi đầu tiên trong danh sách chính là nước đi tốt nhất (thuộc dải PV). Do đó, ta duyệt nước đi này với cửa sổ mở tối đa. Đối với các nước đi còn lại, ta giả định chúng là nước đi kém, nên chỉ dùng một "cửa sổ rỗng" (*Null Window*) cực hẹp để chứng minh chúng không thể vượt qua điểm số hiện tại.

Về phần cài đặt, với nước đi đầu tiên, tìm kiếm với $[-\beta, -\alpha]$; với các nước sau, tìm kiếm với cửa sổ $[-\alpha - 1, -\alpha]$. Chỉ khi kết quả trả về cho thấy nước đi này thực sự tốt hơn điểm α (*fail-high*), ta mới phải tốn công tìm kiếm lại với cửa sổ chuẩn $[-\beta, -\alpha]$.

```

1 pub(super) fn pv_search<const PV: bool>(
2     &mut self,
3     depth: i8,
4     ply: u8,
5     alpha: i32,
6     beta: i32,
7     reductions: i8,
8 ) -> i32 {
9     let mut score = -self.negamax::<false, false>(
10         depth - 1 - reductions,
11         ply + 1,
12         -alpha - 1,
13         -alpha,
14         Default::default(),

```

```

15 );
16
17 // If the search fails and there was depth reduction, we re-search
18 // with the full depth to get a better picture of the position.
19 if alpha < score && reductions > 0 {
20     score = -self.negamax::<false, false>(
21         depth - 1,
22         ply + 1,
23         -alpha - 1,
24         -alpha,
25         Default::default(),
26     );
27 }
28
29 // If the search fails, we need to re-search with the full window to
30 // get the
31 // correct score and PV.
32 if alpha < score && score < beta {
33     score =
34         -self.negamax::<false, PV>(depth - 1, ply + 1, -beta, -alpha,
35         Default::default());
36 }
37 score
38 }
39
40 /// In the negamax function
41 let score = if moves_played == 0 {
42     -self.negamax::<false, PV>(
43         depth - 1 + is_singular as i8,
44         ply + 1,
45         -beta,
46         -alpha,
47         SearchContext::default(),
48     )
49 } else {
50     self.pv_search::<PV>(depth, ply, alpha, beta, reductions)
51 };

```

Listing 4.3: Giải thuật PVS trong Lngine

4.3.7 Các kỹ thuật Mở rộng không gian (Search Extensions)

Các kỹ thuật mở rộng tăng thêm độ sâu vào những thời điểm nhạy cảm để tránh "hiệu ứng chân trời" (*Horizon Effect*).

Mở rộng khi bị chiếu (Check Extensions)

Khi Tướng bị chiếu, mọi nước đi đều là bắt buộc và thường dẫn đến các biến động vật chất lớn. Tạm thời nói rộng độ sâu tìm kiếm giúp engine nhìn xa hơn để tìm ra đường phòng thủ hoặc phát hiện các đòn sát cục.

Trong Lngine, nếu trạng thái bàn cờ hiện tại đang bị chiếu, độ sâu duyệt được cộng thêm 1.

Mở rộng nước đi duy nhất (One-Reply Extensions)

Nếu tại một nút mạng chỉ có đúng 1 nước đi hợp lệ duy nhất, việc duyệt nước đi này không làm bùng nổ số lượng nhánh (hệ số rẽ nhánh bằng 1). Việc đào sâu thêm 1 ply gần như miễn phí về mặt tài nguyên nhưng lại cung cấp tầm nhìn chiến thuật vô giá.

Mở rộng đơn trị (Singular Extensions)

Nếu nước đi từ bảng chuyển vị được đánh giá là cực kỳ tốt, trong khi mọi nước đi khác đều tệ (cách biệt xa), đây là một nước đi "hiển nhiên phải đi". Ta cần mở rộng độ sâu cho nó để chắc chắn rằng đằng sau nước đi hiển nhiên này không ẩn chứa một cái bẫy chiến thuật sâu.

Cài đặt: Ở độ sâu ≥ 8 , Lngine thử duyệt các nước đi thay thế với độ sâu giảm 3 ($R = 3$) và một biên nhỏ $rbeta = score - 2 \times depth$. Nếu các nước đi thay thế này đều thất bại (*fail-low*), nước đi TT chính thức được công nhận là "đơn trị" và được tăng thêm 1 độ sâu khi duyệt.

4.3.8 Các kỹ thuật Cắt tỉa (Pruning) và Giảm độ sâu (Reductions)

Trái ngược với mở rộng, cắt tỉa giúp engine bỏ qua những nhánh vô vọng để dồn sức cho PV.

Giảm độ sâu lặp nội bộ (Internal Iterative Reductions - IIR)

Nếu ta đang ở một độ sâu lớn mà Bảng băm lại không có dữ liệu về thế cờ này, rất có thể đây là một thế trận kém từng bị cắt tỉa ở quá khứ. Thay vì tốn thời gian duyệt sâu ngay lập tức, ta giảm độ sâu đi 1 để quét nhanh, tìm ra nước đi khá nhất rồi mới quyết định duyệt tiếp.

Cắt tỉa nước đi rỗng (Null Move Pruning - NMP)

Lấy ý tưởng "nếu tôi bỏ qua một lượt đi mà thế trận của tôi vẫn đủ mạnh để nghiền nát đối thủ (điểm vượt β), thì tôi không cần tốn thời gian tính toán các nước đi hợp lệ nữa". NMP đóng vai trò cốt lõi trong việc ép cây tìm kiếm nhỏ lại.

Trong cài đặt, Lngine cho đối phương đi hai lần liên tiếp. Độ sâu duyệt được giảm cực mạnh: $R = 3 + \lfloor \frac{depth}{6} \rfloor$. Để tránh lỗi do Zugzwang (đi nước nào cũng thua), NMP chỉ áp dụng khi ta còn quân tấn công. Tại độ sâu lớn (≥ 12), một thuật toán *Verification Search* được gọi để kiểm chứng lại việc cắt tỉa có gây bỏ sót các đòn sát cục ẩn hay không.

Cắt tỉa vô ích ngược (Reverse Futility Pruning - RFP)

Nếu chỉ bằng việc đánh giá tĩnh (*static evaluation*) mà điểm số của ta đã bỏ xa biên β một khoảng an toàn lớn, ta có thể cắt tỉa nhánh ngay tại chỗ mà không cần phải thực hiện bất kỳ nước đi nào.

Trong cài đặt, tại độ sâu < 8 , nếu $Eval - 100 \times (depth - improving) > \beta$, trả về ngay điểm Eval.

Cắt tỉa vô ích (Futility Pruning - FP)

Trái ngược với RFP. Khi ta đang ở tình thế quá lép (điểm Eval cực thấp so với α), và ta không ở thế bị chiếu, những nước đi bình thường không thể nào có phép màu giúp thế cờ lật ngược tình thế. Ta bỏ qua luôn nước này.

Cài đặt: Tại độ sâu ≤ 4 , nếu $\text{Eval} + 100 \times \text{depth} + 50 < \alpha$, các nước quiet sẽ bị bỏ qua (continue).

Cắt tỉa nước đi muộn (Late Move Pruning - LMP)

Tại các độ sâu thấp, sau khi ta đã thử một lượng lớn các nước đi không ăn quân mà không cải thiện được biên α , ta có quyền kết luận rằng các nước đi quiet nằm tít phía dưới bảng xếp hạng (chưa được duyệt) đều là các nước đi vô giá trị.

Cài đặt: Tại độ sâu ≤ 3 , số lượng nước đi quiet được duyệt bị giới hạn bởi công thức: $\text{Limit} = \frac{\text{depth}^2}{1 + \text{improving}} + 8$. Vượt quá giới hạn này, vòng lặp sinh nước đi bị ngắt.

Giảm độ sâu nước đi muộn (Late Move Reductions - LMR)

Với các nước đi không bị cắt tỉa hoàn toàn bởi LMP nhưng nằm trễ trong thứ tự duyệt (chứng tỏ chúng không hấp dẫn), ta vẫn duyệt chúng nhưng với độ sâu giảm bớt. Nếu vô tình chúng trả về điểm fail-high, ta sẽ duyệt lại với độ sâu chuẩn.

Cài đặt: LMR được áp dụng cho các nước đi không ăn quân khi số nước đi đã duyệt ≥ 2 và độ sâu ≥ 3 . Mức giảm R được tính theo công thức phi tuyến:

$$R = 0.75 + \frac{\ln(\text{moves_played}) \times \ln(\text{depth})}{2.3} - \frac{\text{HistoryScore}}{20000} - PV_{\text{flag}}$$

4.3.9 Tìm kiếm tĩnh (Quiescence Search)

Khi cây duyệt đạt đến giới hạn ($\text{depth} \leq 0$), nếu engine gọi ngay hàm đánh giá tĩnh, nó rất dễ rơi vào bẫy ảo ảnh (*Horizon Effect*) – ví dụ một thế cờ mà Xe ta vừa ăn Mã đối phương nhưng ngay sau đó bị đối phương ăn lại Xe. Nhìn bề ngoài ta đang lời Mã, nhưng thực tế là lỗ Xe. Quiescence Search giải quyết điều này bằng cách không dừng lại ngay, mà tiếp tục tạo ra một "cuộc chiến" phụ: chỉ sinh và duyệt các nước đi ăn quân, kéo dài cho đến khi bàn cờ bước vào trạng thái "tĩnh lặng" (không còn nước ăn quân hoặc đổi quân) thì mới trả về điểm số cuối cùng.

Chương 5

Trình diễn và Đánh giá Hệ thống

5.1 Kiểm thử bộ sinh nước đi bằng PERFT

Để đảm bảo tính chính xác tuyệt đối của bộ sinh nước đi (*Move Generator*), vốn là nền tảng quyết định sự đúng luật của toàn bộ hệ thống, chúng tôi sử dụng phương pháp kiểm thử PERFT (*Performance Test*), so sánh với kết quả được công nhận ở trên Internet. Giải thuật PERFT thực hiện duyệt cây trò chơi ở độ sâu d bằng cách sinh tất cả các nước đi hợp lệ từ thế trận ban đầu và đếm tổng số nút lá ở cuối cây.

Kết quả số lượng nút lá đếm được của Lengine được đối chiếu trực tiếp với hệ thống 11 thế cờ chuẩn mực (bao phủ mọi kịch bản góc cạnh của luật cờ tướng) được thừa nhận rộng rãi bởi cộng đồng lập trình cờ. Một phần dữ liệu đối chiếu được trình bày trong Bảng 5.1:

Độ sâu	Số nút	Chiều	Ăn quân	Chiều bí
1	44	0	2	0
2	1,920	6	72	0
3	79,666	384	3,159	0
4	3,290,240	19,380	115,365	0
5	133,312,995	953,251	4,917,734	0
6	5,392,831,844	39,288,662	185,194,510	584

Bảng 5.1: Kết quả kiểm thử chi tiết PERFT của Lengine tại thế cờ khởi đầu

Toàn bộ 11 bài kiểm thử PERFT với các hình thế đặc thù được cấu hình sẵn trong tập tin `perft_positions.txt` đều vượt qua bài test với tỷ lệ chính xác 100% (trạng thái xanh hoàn toàn trên hệ thống `cargo test` thông qua lệnh `cargo run --release --bin perft`). Điều này khẳng định tính đúng đắn dẫn không sai lệch của logic sinh nước đi cho tất cả các quân cờ, bao gồm cả các luật cực kỳ phức tạp như Mã bị cản chân, Pháo lộn ngòi kép hay Tướng lộ mặt.

Hiệu năng và Tốc độ sinh nước đi:

Không chỉ dừng lại ở tính chính xác, Lengine còn thể hiện hiệu năng xử lý ở mức cực hạn nhờ kiến trúc bàn cờ *Bitboard* và kỹ thuật tra cứu bảng tĩnh (*Magic Bitboards*).

Trong bài đo lường thực tế tại thế cờ khởi đầu với độ sâu $d = 6$, hệ thống đã duyệt qua tổng cộng **5 392 831 844** nút. Quá trình này không chỉ đơn thuần đếm tổng số lượng

mà còn bóc tách và phân loại thành công 185 194 510 nước ăn quân, 39 288 662 nước chiếu tướng và 584 thế chiếu bí.

Đáng chú ý, trên Laptop với CPU Intel Core i9-13900HX, xung nhịp đơn là 2.2GHz, Turbo Boost tối đa 5.4GHz, toàn bộ quá trình duyệt hơn 5.3 tỷ trạng thái phức tạp này chỉ mất vồn vẹn **54.65 giây** để hoàn thành. Mặc dù cây trò chơi bùng nổ theo cấp số nhân, tốc độ sinh nước đi trung bình của Lngine vẫn duy trì ổn định ở ngưỡng **98.68 triệu nút mỗi giây**. Con số khổng lồ xấp xỉ 100 triệu nút/giây này là minh chứng rõ ràng nhất cho sự thành công của việc loại bỏ hoàn toàn các vòng lặp quét tia truyền thống, thay thế bằng các phép toán thao tác bit song song hóa trên các thanh ghi 64-bit của CPU.

5.2 Phương pháp đánh giá sức mạnh chơi cờ

Để đánh giá sức mạnh thi đấu của các phiên bản Lngine một cách chính xác, khách quan và khoa học, nhóm phát triển đã xây dựng một hệ thống tự động hóa kiểm thử hiệu năng và ước lượng điểm ELO. Hệ thống này bao gồm các scripts kiểm thử tự động (`run_match.py`, `run_gauntlet.py`, `run_historical_evals.py`) và giao thức phối hợp với trình quản lý giải đấu `sylvan-cli` dành cho cờ tướng.

5.2.1 Scripts và Môi trường Kiểm thử

Mỗi phiên bản engine cờ khi phát triển đều được đưa qua một quy trình kiểm định nghiêm ngặt gồm 3 giai đoạn đối đầu độc lập:

1. **Giai đoạn 1 (Gauntlet Performance):** Phiên bản thử nghiệm sẽ thi đấu một giải Gauntlet với tổng cộng 3000 ván đấu chống lại một tập hợp các đối thủ chuẩn là *Fairy-Stockfish*. *Fairy-Stockfish* được giới hạn sức mạnh thi đấu ở 6 cấp độ ELO cố định: FS-1200, FS-1400, FS-1600, FS-1800, FS-2000, và FS-2200. Với mỗi đối thủ, engine cờ thi đấu đúng 500 ván, luân phiên cầm quân Đỏ (đi trước) và quân Đen (đi sau) để triệt tiêu lợi thế màu quân.
2. **Giai đoạn 2 (Neighbor Match):** Đối đầu trực tiếp giữa phiên bản hiện tại (N) và phiên bản tiền nhiệm gần nhất ($N - 1$) trong 1000 ván đấu để xác định mức cải tiến ELO tương đối trực tiếp.
3. **Giai đoạn 3 (Base Match):** Đối đầu trực tiếp giữa phiên bản hiện tại (N) và phiên bản sơ khai (1.0.0) trong 1000 ván đấu để đo lường tổng tiến bộ lũy kế.

Toàn bộ các trận đấu được chạy với thiết lập kiểm soát thời gian chặt chẽ là 3 giây khởi đầu và tích lũy thêm 0.03 giây sau mỗi nước đi. Môi trường phần cứng được chuẩn hóa để đảm bảo tính đồng đều giữa các lượt chạy. Việc sử dụng cơ sở dữ liệu khai cuộc *Masters Opening Database* với 40,711 ván đấu của các danh thủ cờ tướng giúp tạo ra các vị trí ban đầu ngẫu nhiên, tránh việc engine cờ lặp lại cùng một ván cờ dẫn đến sai lệch kết quả thống kê.

5.2.2 Mô hình Ước lượng ELO bằng MLE

Ước lượng sức mạnh ELO tuyệt đối từ kết quả đối đầu của giải Gauntlet và ELO tương đối từ các trận đối đầu trực tiếp được thực hiện thông qua mô hình toán học Bradley-Terry. Mô hình này giả định rằng mỗi tác tử i sở hữu một tham số sức mạnh ẩn β_i . Xác

suất để tác tử i đánh bại tác tử j được xác định bởi hàm logistic:

$$P(i > j) = \frac{e^{\beta_i}}{e^{\beta_i} + e^{\beta_j}} = \frac{1}{1 + 10^{(ELO_j - ELO_i)/400}}$$

Trong đó, điểm số ELO của tác tử i liên hệ với tham số β_i qua công thức $ELO_i = \beta_i \times \frac{400}{\ln(10)}$. Kết quả của một ván đấu được quy đổi với thắng bằng 1.0, hòa bằng 0.5 và thua bằng 0.

Với tập hợp kết quả của M ván đấu giữa các tác tử, hàm hợp lý (*Likelihood Function*) của mô hình được định nghĩa là:

$$L(\beta) = \prod_{k=1}^M P(w_k > l_k)$$

Trong đó w_k và l_k lần lượt là tác tử thắng (hoặc hòa) và thua trong ván đấu thứ k . Phương pháp ước lượng hợp lý cực đại (*Maximum Likelihood Estimation - MLE*) được sử dụng để tìm bộ tham số ELO tối ưu sao cho cực đại hóa hàm hợp lý trên:

$$ELO^* = \arg \max_{ELO} L(ELO)$$

Sai số tiêu chuẩn (*Standard Error - σ*) và khoảng tin cậy 95% (*95% Confidence Interval*) của giá trị ELO ước lượng được tính toán từ ma trận thông tin Fisher nghịch đảo của hàm hợp lý tại điểm tối ưu, đảm bảo độ tin cậy khoa học cao cho các số liệu thống kê. Do các trận đấu diễn ra độc lập trong các môi trường thi đấu khác nhau, điểm ELO tuyệt đối và chênh lệch ELO tương đối được tối ưu hóa trên các tập mẫu khác nhau, dẫn đến các sai số thống kê nhỏ giữa chúng nhưng vẫn phản ánh tính nhất quán cao về mặt xu hướng.

5.3 Lịch sử phát triển và Tiến trình tăng trưởng Elo

Trải qua chu kỳ phát triển liên tục, Lngine đã có những bước tiến vượt bậc về mặt trình độ chơi cờ. Từ một phiên bản thô sơ ở v1.0.0 chỉ có sức mạnh tương đương người mới học chơi cờ tướng (khoảng 1465 ELO), Lngine v1.9.0 đã đạt ngưỡng 2244 ELO tuyệt đối trên thang đo Gauntlet, vươn lên cấp độ tương đương các kiện tướng hàng đầu trên nền tảng Playstrategy.

5.3.1 Bảng tổng hợp tiến trình 15 phiên bản

Bảng 5.2 trình bày chi tiết sự tiến hóa của Lngine qua các mốc phiên bản bao gồm cả điểm ELO tuyệt đối và các cải tiến kỹ thuật chính:

5.3.2 Phân tích các cột mốc lịch sử

Tiến trình phát triển của Lngine có thể chia làm ba thời kỳ chính với các đặc thù công nghệ riêng biệt:

1. **Thời kỳ Xây dựng nền móng (v1.0.0 – v1.4.0):** Tập trung vào tính đúng đắn của luật cờ tướng và kiến trúc cơ bản. Việc bổ sung *Transposition Table* (v1.2.0) giúp lưu trữ các thế cờ đã duyệt, loại bỏ việc tính toán trùng lặp và đem lại mức

Phiên bản	ELO ước lượng	Diff/Trước	Diff/Gốc	Các cải tiến cốt lõi
1.0.0	1465	N/A	N/A	Sinh nước đi Bitboard, Nega-max Fail-soft, Quiescence Search, Đánh giá vật chất cơ bản.
1.1.0	1460	-9.0	-2.1	Sửa đổi và hoàn thiện luật cờ tướng (Cản Mã, Tướng lộ mặt, Sĩ Tượng cung).
1.2.0	1567	+96.2	+85.0	Triển khai bảng đảo vị trí (<i>Transposition Table - TT</i>).
1.3.0	1572	+20.2	+100.3	Áp dụng kỹ thuật Cửa sổ vọng tướng (<i>Aspiration Window</i>).
1.4.0	1612	+69.4	+145.5	Sắp xếp nước đi với <i>Killer Moves</i> và <i>History Heuristics</i> .
1.5.0	1854	+358.8	+556.7	Tích hợp bảng lịch sử vị trí quân cờ (<i>Piece-Square Tables - PST</i>).
1.6.0	1916	+80.6	+640.0	Tìm kiếm mở rộng: <i>Check, Singular, One-Reply Extensions</i> .
1.6.1	1919	+34.5	+632.9	Tái cấu trúc mã nguồn và sửa các lỗi nhỏ.
1.6.2	1912	-17.7	+576.2	Định dạng phong cách mã nguồn, thêm thông tin PV/seldepth/hashfull.
1.6.3	1902	-4.5	+561.4	Tái cấu trúc, dọn dẹp luồng tìm kiếm, thử nghiệm kiểu dữ liệu i16/i8.
1.6.4	1920	+19.5	+613.0	Khôi phục và tối ưu hóa các thử nghiệm cắt tĩa tìm kiếm.
1.6.5	1936	+33.1	+640.0	Cải thiện cấu trúc dự án, TT trong <i>Quiescence Search</i> .
1.7.0	2027	+152.2	+536.9	<i>Tapered Evaluation, Principal Variation Search</i> (PVS), LMR, <i>Null Move Pruning</i> .
1.8.0	2189	+237.6	+619.4	<i>Mobility Bonus, Defender Bonus</i> , tối ưu hóa tham số bằng <i>Texel Tuning</i> .
1.9.0	2244	+127.4	+651.3	<i>Aspiration Window Fail-high Reductions</i> , RFP, <i>Futility Pruning</i> , LMP, IIR.

Bảng 5.2: Tiến trình phát triển và tăng trưởng sức mạnh ELO của Lngine

tăng ELO ấn tượng (+96.2 ELO). Kỹ thuật sắp xếp nước đi (*Move Ordering*) ở v1.4.0 giúp tối ưu hóa thứ tự nhánh Alpha-Beta, nâng hiệu suất cắt tỉa rõ rệt (+69.4 ELO).

2. **Thời kỳ Ổn định và Dọn dẹp (v1.6.0 – v1.6.5):** Đây là giai đoạn tối ưu mã nguồn. Việc tích hợp các phần mở rộng độ sâu tìm kiếm (*Extensions*) ở v1.6.0 đưa ELO vượt ngưỡng 1900 (+80.6 ELO). Trong các bản phụ v1.6.2 và v1.6.3, ELO suy giảm nhẹ do các thay đổi cấu trúc dữ liệu và thử nghiệm kiểu dữ liệu nông để tiết kiệm bộ nhớ gặp lỗi logic tiềm ẩn hoặc gây chậm tốc độ duyệt. Tuy nhiên, nhóm đã nhanh chóng khắc phục ở v1.6.4 và v1.6.5 bằng việc đưa TT vào *Quiescence Search* (+33.1 ELO).
3. **Thời kỳ Cắt tỉa nâng cao và Tự động hóa tối ưu (v1.7.0 – v1.9.0):** Trực tiếp đưa engine cờ vượt qua mốc ELO 2000 và 2200. Các thuật toán cắt tỉa hiện đại như *Null Move Pruning* và *Late Move Reductions* (v1.7.0) cho phép engine bỏ qua các nhánh tìm kiếm vô vọng, tập trung tài nguyên vào các nhánh PV then chốt (+152.2 ELO). Phiên bản v1.9.0 hoàn thiện hóa kỹ thuật cắt tỉa với một loạt cải tiến nâng cao như *Reverse Futility Pruning* (RFP) và *Late Move Pruning* (LMP), tăng thêm +127.4 ELO.

5.3.3 Phân tích các Bước Nhảy ELO Lớn

Trong lịch sử phát triển, có hai bước nhảy ELO đột phá, đóng góp phần lớn sức mạnh cho Lngine:

- **Bước nhảy 1 (v1.5.0: Piece-Square Tables – PST, +358.8 ELO):** Đây là mức tăng trưởng ELO đơn lẻ lớn nhất. Phiên bản v1.0.0 đến v1.4.0 chỉ đánh giá thế trận dựa trên tổng giá trị vật chất của các quân cờ hiện diện trên bàn (ví dụ Xe = 900, Pháo = 450...). Điều này khiến tác tử chơi vô hồn, không biết đưa Xe chiếm lộ đẹp, không biết đẩy Tốt qua sông hay không giữ vững Tượng ở trung lộ. PST đã gán giá trị định vị cho từng quân cờ tại từng tọa độ cụ thể. Sự thay đổi từ hàm đánh giá tĩnh thô sơ sang hàm đánh giá kết hợp vị trí đã thay đổi hoàn toàn chất lượng nước đi, làm tăng vọt sức mạnh chơi cờ của Lngine thêm hơn 350 ELO chỉ trong một bản cập nhật.
- **Bước nhảy 2 (v1.8.0: Texel Tuning & Evaluation Bonuses, +237.6 ELO):** Mặc dù PST giúp định vị quân cờ tốt, các giá trị trong bảng PST và trọng số đánh giá ban đầu được xác định thủ công dựa trên cảm quan của con người, dẫn đến sự thiếu chính xác và mất cân đối ở các giai đoạn tàn cuộc. Thuật toán *Texel Tuning* sử dụng phương pháp tối ưu hóa phi tuyến tính để tự động căn chỉnh các tham số dựa trên dữ liệu hàng ngàn ván đấu thực tế. Kết hợp với việc bổ sung điểm thưởng cơ động (*Mobility Bonus*) và quân phòng thủ bảo vệ Tượng (*Defender Bonus*), Lngine đã học được cách phối hợp quân cờ phức tạp, giúp tăng vượt bậc 237.6 ELO trực tiếp khi đối đầu phiên bản v1.7.0.

Sự thành công của hai bước nhảy này chứng minh rằng trong thiết kế engine cờ tướng, việc cải tiến độ chính xác của hàm đánh giá tĩnh (*Evaluation Function*) luôn đem lại hiệu quả tăng sức mạnh ELO vượt trội so với việc chỉ tập trung tối ưu hóa chiều sâu tìm kiếm (*Search Function*).

5.4 Đánh giá chi tiết phiên bản mới nhất v1.9.0

Phiên bản Lngine v1.9.0 là đỉnh cao công nghệ hiện tại của dự án, tích hợp đầy đủ các giải thuật tìm kiếm cắt tỉa hiện đại nhất và hàm đánh giá được tinh chỉnh tối ưu. Để có cái nhìn toàn diện về sức mạnh của phiên bản này, chúng tôi phân tích kết quả thu được từ ba giai đoạn kiểm thử độc lập.

5.4.1 Đánh giá Gauntlet (Phase 1)

Trong giải Gauntlet gồm 3000 ván đấu với 6 cấp độ Fairy-Stockfish, Lngine v1.9.0 đạt điểm ELO ước lượng tối ưu là **2244 ELO** với sai số tiêu chuẩn cực nhỏ ± 10.9 ELO. Khoảng tin cậy 95% dao động trong ngưỡng từ 2222 đến 2265 ELO. Tỷ lệ ghi điểm tổng thể cực kỳ ấn tượng đạt **87.6%** với 2597 trận thắng, 65 trận hòa và 338 trận thua.

Bảng 5.3 chi tiết hóa kết quả đối đầu của Lngine v1.9.0 trước từng đối thủ baseline:

Đối thủ	ELO	Số ván	Thắng	Hòa	Thua	Tỷ lệ điểm	ELO ước lượng
FS-1200	1200	500	493	2	5	98.8%	1966 ELO
FS-1400	1400	500	497	0	3	99.4%	2288 ELO
FS-1600	1600	500	488	4	8	98.0%	2276 ELO
FS-1800	1800	500	473	7	20	95.3%	2323 ELO
FS-2000	2000	500	381	17	102	77.9%	2219 ELO
FS-2200	2200	500	265	35	200	56.5%	2245 ELO
—		3000	2597	65	338	87.6%	2244 ELO

Bảng 5.3: Kết quả chi tiết giải đấu Gauntlet của Lngine v1.9.0

Số liệu chỉ ra rằng Lngine v1.9.0 hoàn toàn áp đảo các đối thủ từ FS-1800 trở xuống khi duy trì tỷ lệ điểm trên 95%. Trước đối thủ mạnh nhất là FS-2200, Lngine vẫn giành thắng lợi chung cuộc với tỷ lệ điểm là 56.5% (265 thắng, 35 hòa, 200 thua), qua đó khẳng định trình độ thi đấu vượt mức 2200 ELO một cách rõ ràng.

5.4.2 Đối đầu trực tiếp phiên bản tiền nhiệm v1.8.0 (Phase 2)

Trận đấu Neighbor Match giữa Lngine v1.9.0 và v1.8.0 (1000 ván) kết thúc với phần thắng áp đảo nghiêng về phiên bản mới:

- **Số ván thắng:** 522 ván.
- **Số ván hòa:** 307 ván.
- **Số ván thua:** 171 ván.
- **Tỷ lệ điểm:** 67.5%.
- **Chênh lệch ELO ước lượng:** $+127.4 \pm 9.5$ ELO
- **Khoảng tin cậy 95%:** [108.8, 145.9] ELO.
- **Khả năng vượt trội (Likelihood of Superiority):** 100.0%.

Việc giành tới 52.2% số ván thắng và chỉ để thua 17.1% trước một phiên bản rất mạnh như v1.8.0 chứng minh các cải tiến cắt tỉa tìm kiếm nâng cao (như *Reverse Futility Pruning*, *Late Move Pruning* và *Internal Iterative Reductions*) đã giúp Lngine v1.9.0 tiết kiệm hàng triệu nút tìm kiếm, đi sâu hơn vào các phương án chiến thuật sắc bén.

5.4.3 Đối đầu trực tiếp phiên bản gốc v1.0.0 (Phase 3)

Trong trận đấu Base Match chống lại phiên bản sơ khai Lngine v1.0.0 (1000 ván), sự chênh lệch đẳng cấp được thể hiện một cách tuyệt đối:

- **Số ván thắng:** 955 ván.
- **Số ván hòa:** 44 ván.
- **Số ván thua:** 1 ván duy nhất.
- **Tỷ lệ điểm:** 97.7%.
- **Chênh lệch ELO ước lượng:** $+651.3 \pm 26.2$ ELO.

Kết quả thắng 95.5% và chỉ thua duy nhất 1 ván trong tổng số 1000 ván đấu chứng minh Lngine v1.9.0 đã hoàn toàn lột xác. Phiên bản gốc v1.0.0 với bộ thuật toán Negamax thô sơ và hàm đánh giá vật chất đơn giản hoàn toàn không có khả năng chống đỡ trước hệ thống tìm kiếm sâu sắc và định vị quân cờ thông minh của phiên bản mới.

5.4.4 Đặc điểm Thống kê Đáng Chú Ý

Phân tích sâu dữ liệu thống kê từ hàng ngàn trận đấu mang lại các khám phá khoa học thú vị về hành vi của tác tử Lngine v1.9.0:

1. **Sự biến động của Tỷ lệ Hòa:** Tỷ lệ hòa có xu hướng tăng tỷ lệ thuận với trình độ của đối thủ. Trong giải Gauntlet, tỷ lệ hòa trước FS-1200 gần như bằng không (0.4%) và trước FS-1400 là 0%, nhưng đã tăng lên 3.4% trước FS-2000 và đạt 7.0% trước FS-2200. Trong trận đối đầu đồng đẳng cao nhất giữa v1.9.0 và v1.8.0, tỷ lệ hòa vọt lên mức **30.7%**. Điều này phản ánh quy luật tự nhiên của cờ tướng: khi trình độ của cả hai bên càng cao và ít mắc sai lầm sơ đẳng, khả năng ván đấu kết thúc với kết quả hòa càng lớn.

2. Phân bố độ dài ván đấu:

- Trong trận đối đầu với v1.0.0, số nước đi trung bình của mỗi ván đấu chỉ là **60.5 plies** (khoảng 30 nước đi), trung vị là 58 plies. Trận đấu diễn ra vô cùng nhanh gọn vì Lngine v1.9.0 dễ dàng khai thác các sơ hở định vị và dứt điểm đối thủ ở trung cuộc.
- Ngược lại, trong trận đối đầu với v1.8.0, số nước đi trung bình kéo dài đến **159.8 plies** (khoảng 80 nước đi), trung vị là 133 plies, cá biệt có ván đấu kéo dài kỷ lục tới 519 plies. Trận đấu diễn ra giằng co quyết liệt, đòi hỏi kỹ năng cờ tàn điều luyện từ cả hai phía.

3. **Lợi thế đi trước của quân Đỏ:** Trong giải Gauntlet của Lngine v1.9.0, tỷ lệ ghi điểm khi cầm quân Đỏ đi trước là **87.7%** (1301 thắng, 29 hòa, 170 thua) và cầm quân Đen đi sau là **87.6%** (1296 thắng, 36 hòa, 168 thua). Sự chênh lệch tỷ lệ điểm chỉ là 0.1%. Đây là một con số đặc biệt ấn tượng, chỉ ra rằng Lngine v1.9.0 đã giải quyết xuất sắc bài toán phòng thủ khi cầm quân Đen đi sau dưới áp lực khai cuộc của đối thủ, đạt độ cân bằng chiến thuật gần như hoàn hảo giữa hai màu quân.

Chương 6

Kết luận và Hướng phát triển

6.1 Các vấn đề thách thức và Giải pháp

Trong quá trình thiết kế và phát triển engine cờ tướng Lingine, nhóm nghiên cứu đã đối mặt và giải quyết thành công một số thách thức kỹ thuật lớn:

1. Hiệu ứng chân trời (Horizon Effect):

- *Thách thức:* Xảy ra khi engine phát hiện một đòn chiến thuật bất lợi (ví dụ bị bắt Xe) ở giới hạn độ sâu tìm kiếm. Để trì hoãn việc này ra sau “chân trời” tìm kiếm, engine có thể thực hiện các nước đi thí quân vô nghĩa nhằm kéo dài độ sâu, dẫn đến đánh giá sai lầm nghiêm trọng. *Giải pháp:* Triển khai bộ tìm kiếm tĩnh Quiescence Search kết hợp với kiểm tra chiếu tướng mở rộng (Check Extensions). Quiescence Search tiếp tục duyệt tất cả các nước đi ăn quân cho đến khi thế trận hoàn toàn bình ổn tĩnh, đảm bảo điểm số trả về phản ánh chính xác thực tế thế cờ.

2. Tranh chấp đồng bộ hóa trong giao thức UCI:

- *Thách thức:* Lệnh ngắt `stop` từ người dùng có thể gửi đến bất kỳ lúc nào qua luồng vào chuẩn khi engine đang tính toán sâu. Nếu sử dụng các cơ chế đồng bộ luồng chặn hoặc khóa Mutex truyền thống sẽ gây trễ hoặc làm treo toàn bộ ứng dụng.
- *Giải pháp:* Áp dụng biến nguyên tử `AtomicBool` chia sẻ giữa luồng đọc lệnh và luồng tìm kiếm. Thao tác đọc ghi bất đồng bộ không chặn giúp dừng tìm kiếm tức thời trong thời gian dưới 1 mili-giây mà không gây ra hiện tượng nghẽn luồng.

6.2 Công nghệ và Tài nguyên sử dụng

Dự án Lingine được xây dựng và kiểm thử dựa trên các công nghệ và tài nguyên mã nguồn mở hiện đại:

- **Rust Toolchain:** Trình biên dịch `rustc` và công cụ quản lý dự án `cargo`. Chúng tôi tận dụng tối đa các tối ưu hóa biên dịch của Cargo thông qua cấu hình biên dịch `opt-level = 3` và kích hoạt các chỉ thị tập lệnh CPU vector hiện đại (`-C target-cpu=native`).

- **Thư viện `arrayvec`:** Cho phép lưu trữ danh sách nước đi (`MoveList`) trong bộ nhớ Stack cố định thay vì bộ nhớ Heap động, tránh overhead cấp phát bộ nhớ khi sinh hàng triệu nước đi mỗi giây.
- **Python Suite:** Các kịch bản Python tự động thiết lập môi trường (`setup_tools.py`) và tổ chức đấu giải ước lượng Elo (`run_match.py`, `run_gauntlet.py`, `run_historical_evals.py`).
- **Trình quản lý `Sylvan`:** Công cụ *Sylvan* được sử dụng làm giao diện người dùng đồ họa (GUI) để kiểm thử thủ công và *Sylvan-CLI* đóng vai trò làm bộ điều phối giải đấu thông qua giao thức UCI chuẩn mực.
- **Các engine cờ tướng đối chiếu:** *Fairy-Stockfish* và *Pikafish* làm các đối thủ có giới hạn sức mạnh chuẩn để chạy giải đấu ước lượng ELO và là công cụ kiểm thử tính đúng đắn của luật chơi.
- **Dự án cờ vua mã nguồn mở bằng Rust:** Tham khảo cấu trúc kiến trúc mã nguồn tối ưu hiệu năng từ hai dự án *Reckless* và *Viridithas* viết bằng Rust.

6.3 Kết luận

Dự án nghiên cứu thiết kế và triển khai engine cờ tướng Lngine đã hoàn thành đầy đủ tất cả các mục tiêu đề ra ban đầu:

- Xây dựng thành công một engine cờ tướng hoàn chỉnh bằng ngôn ngữ Rust, đạt độ chính xác luật chơi tuyệt đối được xác minh qua các bộ kiểm thử PERFTEST chuẩn quốc tế.
- Triển khai cấu trúc dữ liệu Bitboard 128-bit kết hợp với Magic Bitboards giúp tối ưu hóa cực đoan tốc độ sinh nước đi, đạt hiệu suất xử lý hàng triệu nút trạng thái trên giây.
- Tích hợp hàm đánh giá tĩnh tapered score hiện đại, tự động tối ưu hóa các tham số thông qua thuật toán Texel Tuning.
- Triển khai bộ thuật toán tìm kiếm nâng cao Negamax tích hợp PVS, bảng băm TT và các kỹ thuật tỉa nhánh hiện đại giúp tăng độ sâu chiến thuật và đạt mức Elo thi đấu ước lượng tối đa khoảng 2244.

6.4 Hướng phát triển tương lai

Để tiếp tục nâng cao sức mạnh và tính thực tiễn của engine, nhóm phát triển đề xuất hai hướng nghiên cứu trọng tâm tiếp theo:

6.4.1 Tăng tính đa dạng trong lối chơi bằng Softmax Sampling

Hiện tại, engine luôn chọn nước đi tốt nhất tuyệt đối có điểm số cao nhất (**bestmove**). Trong đấu tập hoặc giai đoạn khai cuộc, điều này làm lối chơi của engine dễ bị bắt bài và rập khuôn.

Hướng cải tiến là áp dụng lấy mẫu xác suất Softmax dựa trên điểm số đánh giá $Q(s, a)$ của các nước đi khả dĩ:

$$P(a_i|s) = \frac{e^{Q(s, a_i)/\tau}}{\sum_j e^{Q(s, a_j)/\tau}}$$

Trong đó τ là tham số nhiệt độ (Temperature). Khi $\tau \rightarrow 0$, engine chọn nước đi tốt nhất; khi τ lớn hơn, engine sẽ ngẫu nhiên chọn các nước đi tốt khác nhau với xác suất tương ứng, tạo nên sự phong phú và tự nhiên trong phong cách chơi cờ giống con người.

6.4.2 Tích hợp Mạng nơ-ron NNUE (Efficiently Updatable Neural Networks)

Hàm đánh giá hiện tại của Lngine là hàm tuyến tính kết hợp Material và PST. Mặc dù chạy nhanh nhưng hàm tuyến tính bỏ sót rất nhiều mối quan hệ phi tuyến tính phức tạp giữa các quân cờ (ví dụ mối tương tác giữa Xe, Pháo, Mã khi phối hợp tấn công).

Hướng phát triển đột phá là tích hợp cấu trúc mạng nơ-ron NNUE dạng nông (thường gồm một lớp ẩn kích thước 256 hoặc 512). Điểm đặc biệt của NNUE là lớp mạng đầu vào được cập nhật vi phân vô cùng nhanh chóng ngay trong quá trình thực hiện nước đi (do_move/undo_move) dựa trên sự dịch chuyển của quân cờ, giúp giữ nguyên tốc độ NPS cao của engine trong khi mang lại độ chính xác đánh giá thể trận vượt trội tương đương các đại kiện tướng quốc tế.

Tài liệu tham khảo

1. **Đại học Bách khoa Hà Nội**, *Bài giảng Trí tuệ Nhân tạo IT3160*, TS. Ngô Văn Linh, Trường Công nghệ Thông tin và Truyền thông.
2. **Russell, S. và Norvig, P.**, *Artificial Intelligence: A Modern Approach*, Prentice Hall, 4th Edition, 2020 (Tài liệu về mô hình tác tử PEAS và thuật toán Alpha-Beta).
3. **Chess Programming Wiki (CPW)**, <https://www.chessprogramming.org/> (Tài liệu tham khảo chi tiết về Bitboards, Magic Bitboards, Transposition Table, PVS, NMP, LMR, và Singular Extensions).
4. **Steve J. Edwards**, *An Introduction to Zobrist Hashing*, 1996 (Cơ chế sinh khóa và cập nhật vi phân mã băm).
5. **Peter Bradley và John Terry**, *Rank Analysis of Incomplete Block Designs: I. The Method of Paired Comparisons*, Biometrika, 1952 (Mô hình Bradley-Terry dùng ước lượng sức mạnh Elo).
6. **Peter Österlund**, *Texel Tuning Method*, <https://github.com/peterosterlund/texel> (Phương pháp tối ưu hóa tham số đánh giá thế cờ bằng hồi quy logistic).
7. **The Rust Programming Language**, Steve Klabnik và Carol Nichols, No Starch Press, 2018 (Tài liệu hướng dẫn tối ưu hiệu năng và an toàn bộ nhớ trong Rust).
8. **Fairy-Stockfish Project**, <https://github.com/ianfab/Fairy-Stockfish> (Công cụ đối thủ chuẩn dùng cho cờ tướng và các biến thể cờ vua).
9. **Sylvan Project**, Wilbert Lee và các cộng sự, <https://github.com/EterCyber/Sylvan> (Ứng dụng giao diện người dùng và trình quản lý giải đấu UCI cho engine cờ).
10. **Reckless Chess Engine**, <https://github.com/codedeliveryservice/Reckless> (Engine cờ vua viết bằng Rust dùng tham khảo kiến trúc và quản lý trạng thái).
11. **Viridithas Chess Engine**, Cosmo Bobak, <https://github.com/cosmobobak/viridithas> (Engine cờ vua mã nguồn mở viết bằng Rust làm mẫu thiết kế cấu trúc dữ liệu).
12. **Pikafish Project**, <https://github.com/official-pikafish/Pikafish> (Engine cờ tướng mạnh mẽ phát triển từ Stockfish hỗ trợ so khớp tính đúng đắn của luật cờ).